MACHINE LEARNING · UNSUPERVISED LEARNING

Clustering Hyperparameter Tuning & Grid Search

K-Means · Hierarchical (Agglomerative) · DBSCAN

A complete tutorial from first principles to pro level — with real experiments, visuals and working code

1 Basic — what & why → 2 Intermediate — hyperparameters → 3 Advanced — grid search → 4 Pro — speed & pitfalls

---

Every picture in this PDF was generated by running scikit-learn code (v1.9) — the numbers you see are real results.

How to use this tutorial

Your notes covered three clustering algorithms, each with a hyperparameter table, deep-dives, efficiency tips and a grid-search template. They were repeated in places and had no pictures. This tutorial **keeps every point from your notes**, removes the repetition by merging duplicate explanations, and adds the missing basics, diagrams and measured experiments.

🔑 The learning path

Level	Goal	You will be able to…
1 Basic	Understand the idea	Explain what each algorithm does using a simple picture
2 Intermediate	Know every knob	Say what each hyperparameter does and its typical values
3 Advanced	Tune it properly	Run a correct grid search and read the result
4 Pro	Make it fast & reliable	Speed up big data, avoid traps, choose the right algorithm

Contents

Part 0 · Foundations
Clustering, hyperparameters, distance, scaling, silhouette, grid search

Part 1 · K-Means
Algorithm → 7 hyperparameters → deep dive → GridSearchCV pipeline → 5 speed-ups

Part 2 · Hierarchical (Agglomerative)
Dendrogram → linkage, metric, n_clusters/threshold → manual grid search → 4 speed-ups

Part 3 · DBSCAN
Core/border/noise → 8 hyperparameters → eps & min_samples → grid search → 4 speed-ups + HDBSCAN

Part 4 · Pro Playbook
Algorithm chooser, side-by-side cheat sheet, mistakes, practice exercises

Appendix
Glossary and your three complete code templates

PART 0

Foundations

LEVEL 1 · BASIC

0.1 What is clustering?

Clustering means automatically grouping similar things together *without being told the right answer*. Because there are no answer labels (no "y"), it is called **unsupervised learning**.

> 🍕 **Everyday analogy**
>
> Imagine dumping a box of mixed buttons on a table. Nobody tells you the categories, but you naturally make piles: big red ones, small white ones, shiny gold ones. You just did clustering — the "distance" you used was how similar the buttons look.



Left: raw points with no labels. Right: a clustering algorithm discovers three groups (colours are the cluster labels).

0.2 Parameters vs hyperparameters

	Parameter	Hyperparameter
Who sets it?	The algorithm learns it from data	**You** choose it before training
K-Means example	Where the centroids end up	`n_clusters`, `init`, `n_init`
DBSCAN example	Which points get which label	`eps`, `min_samples`

Hyperparameter tuning = trying different settings and keeping the best. **Grid search** = trying *every combination* from a list you provide, like testing every cell of a spreadsheet grid.

🧪 Grid size example

K-Means with `n_clusters` in 6 values × `init` in 2 × `n_init` in 2 × `algorithm` in 2 = 6×2×2×2 = **48 combinations** — and with `cv=3` each is trained 3 times (144 fits). Grids grow fast, so only tune the parameters that matter.

0.3 Distance — the heart of clustering

All three algorithms decide "are these two points similar?" using **distance**. Small distance = similar. Two consequences you must remember for the whole tutorial:

- **Scale matters.** If one column has huge numbers (income: 30,000–150,000) and another has small numbers (age: 18–80), the big column dominates every distance. Fix: `StandardScaler` (makes every column have mean 0 and standard deviation 1).
- **Too many columns blur distance.** In very high dimensions all points look equally far apart (the "curse of dimensionality"). Fix: PCA / UMAP first.

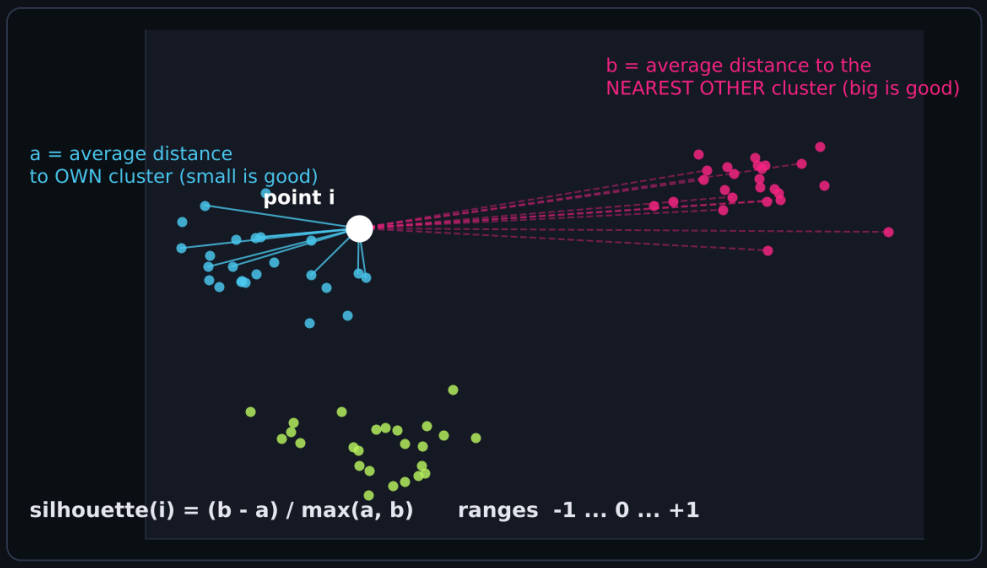


Same data, same K-Means. Without scaling, income (big numbers) decides everything and the age groups are missed (ARI≈-0.0). With StandardScaler the two age groups are found perfectly (ARI=1.0). ARI = Adjusted Rand Index: 1 = perfect match with truth, 0 = random.

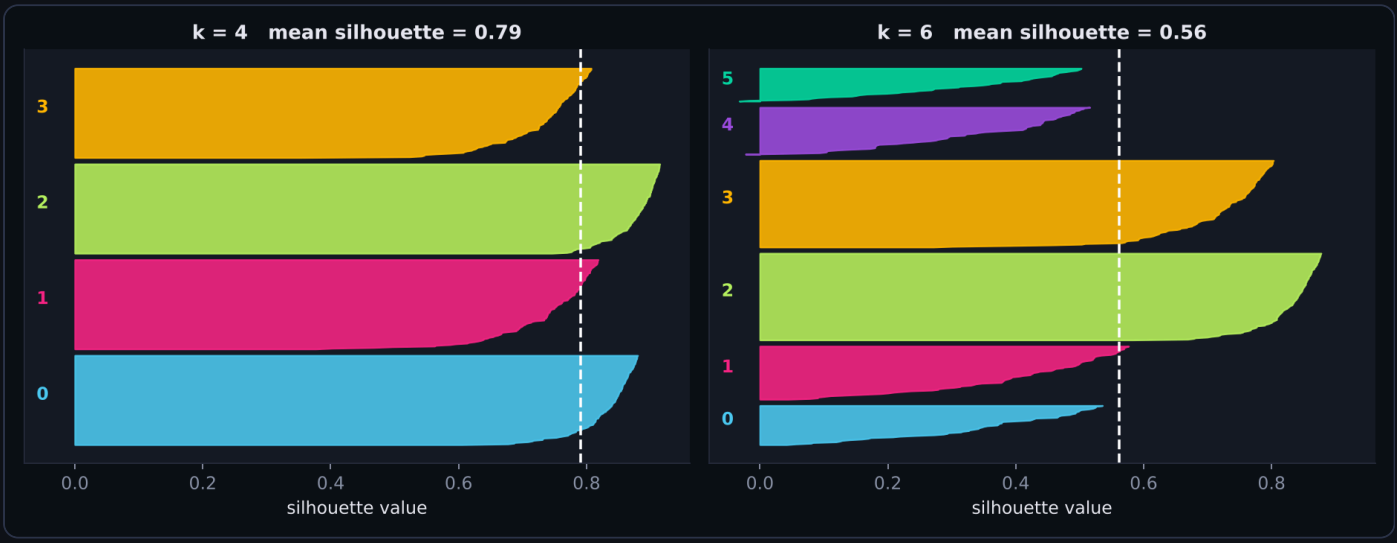
0.4 How do we score a clustering with no answer key?

Supervised models use accuracy. Clustering has no labels, so we use **internal metrics** that measure how tight and well-separated the groups are.

Metric	Idea	Better is…	Notes
Inertia (SSE)	Sum of squared distances from points to their centroid	Lower	**Always** decreases when k grows, so it cannot choose k by itself (use the "elbow" look)
Silhouette score	How much closer a point is to its own cluster than to the next-nearest cluster	Higher (−1…+1)	Needs ≥ 2 clusters. Main score used in this tutorial
Calinski-Harabasz	Ratio of between-cluster spread to within-cluster spread	Higher	Cheap to compute; alternative to silhouette



Silhouette of one point: a = how far on average from its own cluster-mates; b = how far on average from the closest other cluster. Score near +1 means “clearly in the right group”, near 0 means “on the border”, negative means “probably in the wrong group”.



A “silhouette plot”: each coloured blade is one cluster, each thin line one point. Wide, long blades above the dashed average line are healthy. At k=6 the mean drops because true groups were split.

> **⚠️ Silhouette has a bias**
>
> It rewards compact, round clusters. It can prefer a *wrong but round* answer over a *right but curved* one. You will see a real example in Part 2 (moons). Always look at the picture too.

0.5 Two special words: inductive vs transductive

	Has `.predict()` for new data?	Algorithms	Can use `GridSearchCV`?
Inductive	Yes — learns a rule, can place new points	K-Means	**Yes** (with custom scorer)
Transductive	No — only groups the exact data it was fitted on	Agglomerative, DBSCAN	**No** → use a manual loop with `ParameterGrid`

This single fact explains why the three grid-search templates in this tutorial look different.

PART 1

K-Means Clustering

LEVEL 1 · BASIC — the idea

1.1 K-Means in one sentence

Pick **k** centre points (**centroids**), assign every data point to its nearest centroid, move each centroid to the average of its points, and repeat until nothing changes.

> 🍕 **Analogy: opening k pizza shops**
>
> You will open 3 pizza shops in a city. Start with 3 random locations. Every customer goes to the closest shop. Then each shop moves to the middle of *its own* customers. Customers re-choose, shops move again… After a few rounds the shops stop moving. Those final positions are the centroids and each shop's customers are one cluster.



The K-Means loop on real data (stars = centroids). Steps 2–5 repeat until the assignment stops changing.

1.2 The algorithm (Lloyd's algorithm)

1. **Initialize**: choose k starting centroids.
2. **Assign**: each point joins the cluster of its closest centroid.

3. **Update**: move each centroid to the mean of its points.

4. **Repeat** steps 2–3 until centroid movement is below `tol` or `max_iter` iterations are reached.

> 🔑 **Cost**
>
> Runtime grows linearly with the number of clusters: **O(k · n · T)** (k clusters, n points, T iterations; in more detail also multiplied by the number of columns). That is why K-Means is fast and suits big data.

`LEVEL 2 · INTERMEDIATE — every hyperparameter`

1.3 Complete hyperparameter table

Scikit-Learn's K-Means has several hyperparameters that control how the algorithm **initializes**, **converges** and **builds clusters**. Because K-Means is unsupervised, grid searching requires evaluating against metrics like the **Silhouette Score** rather than default accuracy.

Hyperparameter	Description	Preferred grid search values
`n_clusters`	The number of clusters (k) to form and centroids to generate.	`[2, 3, 4, 5, 6, 7, 8, 9, 10]`
`init`	The strategy for choosing the initial centroid locations.	`['k-means++', 'random']`
`n_init`	The number of times the algorithm runs independently with different random seeds.	`[10, 20, 30]` (or `'auto'` in newer versions)
`max_iter`	The maximum number of iterations allowed for a single run before stopping.	`[300, 500]` (rarely needs tuning)
`tol`	The threshold for declaring convergence based on centroid movement.	`[1e-4, 1e-3]` (rarely needs tuning)
`algorithm`	The mathematical implementation used to compute distances (`'lloyd'` or `'elkan'`).	`['lloyd', 'elkan']`
`random_state`	The seed for reproducible random generation.	`[42]` (fix to a single integer, do not tune)

> 🔑 **The big picture**
>
> While algorithms like Random Forest need a massive grid, K-Means performance is dictated by just **three** hyperparameters: `n_clusters`, `init` and `n_init`. The other four are "set and forget".

1.4 The four "set and forget" parameters

Parameter	Plain-English meaning	When to touch it
`max_iter`	Safety limit on loops per run (default 300). Like "stop after 300 rounds even if still moving".	Only if the run prints a ConvergenceWarning → raise to 500+.

Parameter	Plain-English meaning	When to touch it
`tol`	“Stop when centroids move less than this.” Smaller = stricter, slower.	Almost never. 1e-4 is fine.
`algorithm`	`'lloyd'` = classic. `'elkan'` = same result, but skips distance calculations using the triangle inequality (see 1.9).	Try both if speed matters. Result is the same; only time (and memory) differs.
`random_state`	Fixes the random seed so you get the same answer every run.	Fix it (e.g. 42) so experiments are reproducible. **Never tune it** — that would be tuning luck.

LEVEL 3 · ADVANCED — deep dive on the 3 key parameters

1.5 `n_clusters` — number of clusters

⚙️ How it works

It dictates the exact number of groupings (k) the algorithm is forced to partition the data into. The algorithm will generate **exactly k centroids** — even if the data has no real groups.

✅ How it helps

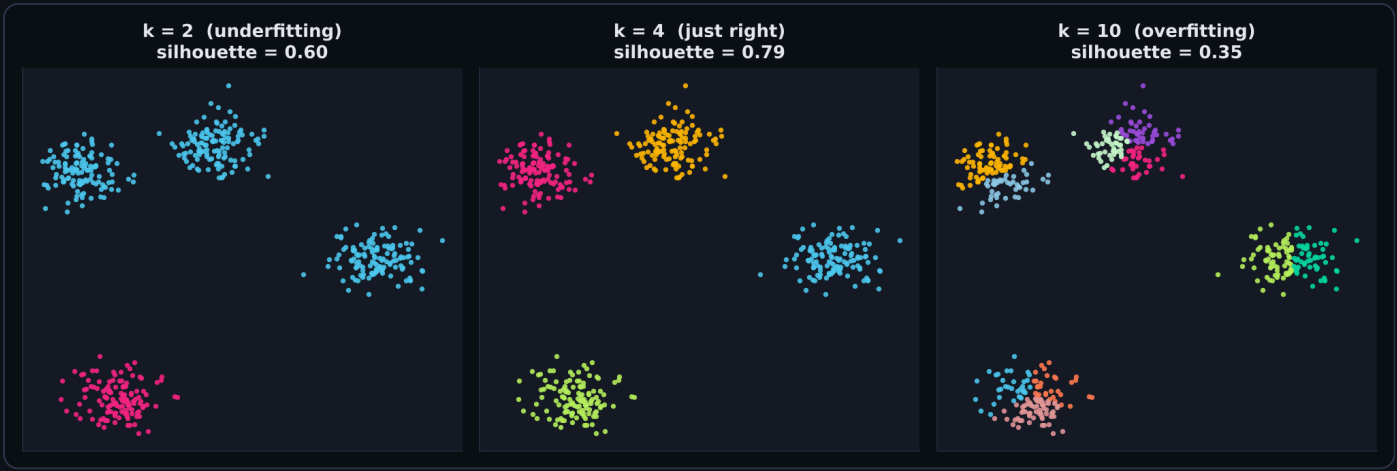
It defines the **granularity** of your analysis. Clustering customers: `n_clusters=3` might give broad categories (low, medium, high spenders), while `n_clusters=10` reveals niche micro-segments.

📉 How it affects the model

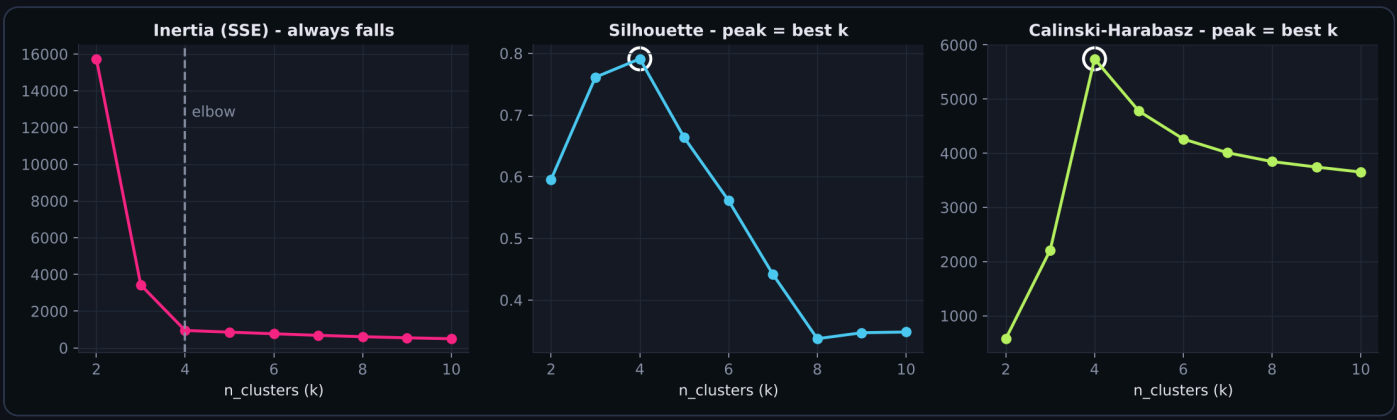
Too low → **“underfitting”**: fundamentally different points are lumped together. Too high → **“overfitting”**: natural groups are split into fragmented, meaningless pieces. Computationally, runtime scales linearly with k: **O(k·n·T)**.

🔍 Grid search strategy

Grid `[2…10]`. **You cannot use K-Means’ default score (inertia / SSE) to grid-search n_clusters**, because inertia mathematically *always decreases* as k increases. Use a custom scorer like `silhouette_score` or `calinski_harabasz_score` to find the mathematically optimal k.



The same data with k = 2, 4, 10. The data really has 4 blobs: k=2 merges blobs, k=10 chops blobs into slivers. The silhouette peaks at the right k.



Why inertia can’t pick k: it keeps falling (left, you can only look for an “elbow”). Silhouette and Calinski-Harabasz both peak at k=4 — the true number of blobs.

> **🚀 Business logic beats math**
>
> When running grid search for unsupervised models, always verify that the scoring metric aligns with your business logic — mathematical tightness does not always equal real-world usefulness. If marketing can only run 3 campaigns, k=3 may be more useful than a mathematically “best” k=7.

1.6 `init` — initialization strategy

> **⚙️ How it works**
>
> K-Means starts by placing centroids in the data space. `init='random'` places them strictly at random among the data points. `init='k-means++'` places the first centroid randomly, then places each following centroid **as far from the existing centroids as possible**, chosen with probability proportional to squared distance.

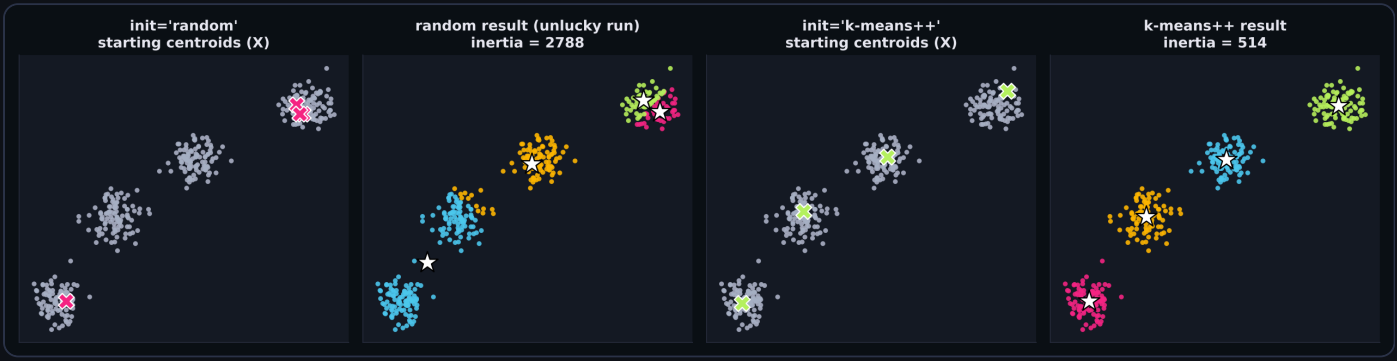
> **✅ How it helps**
>
> Standard random initialization often puts centroids too close together, forcing massive realignments. k-means++ spreads them out logically from step one.

📉 How it affects the model

k-means++ significantly **reduces the iterations** needed to converge and, more importantly, dramatically lowers the chance of getting trapped in a **“poor local minimum”** — a mathematically valid but practically useless grouping.

🔍 Grid search strategy

Test `['k-means++', 'random']`. k-means++ is vastly superior in most cases, but testing both can occasionally reveal quirks in highly dimensional, noisy datasets where random initialization stumbles into a better start.



The worst of 200 single random starts (left two): three centroids start inside the right-most blob, so two blobs end up sharing one centroid and another blob is split (inertia 2788). k-means++ (right two) spreads the starting points and finds the clean answer (inertia 514). Lower inertia = better.

🍕 Analogy

Random start = dropping 4 flags blindly on a map; two may land in the same village. k-means++ = “place the next flag far from the flags already placed”.

1.7 `n_init` — number of initializations

⚙️ How it works

K-Means relies on random starting positions, so one run might be bad purely by bad luck. `n_init` runs the whole clustering from scratch several times (default 10 in older versions), compares all finished models and returns only the one with the **lowest inertia** (tightest clusters).

✅ How it helps

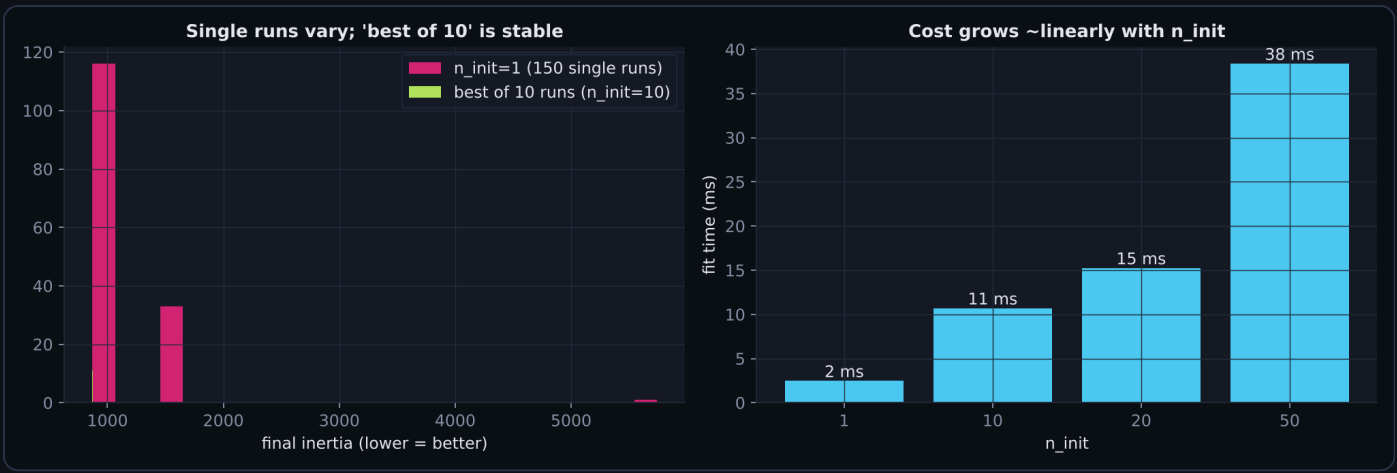
An automated safety net: a single unlucky initialization cannot ruin the final model.

📉 How it affects the model

Higher n_init → more **stability and consistency** across runs. But cost is strictly multiplicative: if one run takes 5 s, `n_init=20` takes 100 s.

🔍 Grid search strategy

Try `[10, 20, 50]`. For a massive dataset where runtime is an issue, tune downward. For a small dataset with complex cluster shapes, tune up to 50 so the search of the feature space is exhaustive.



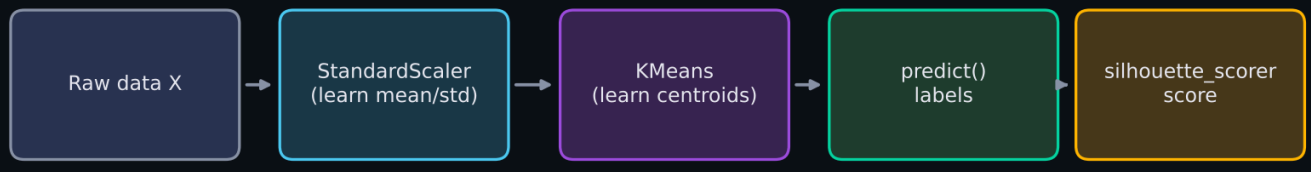
Left: 150 single random-init runs on the same data end with inertia anywhere from 871 (great) to 5739 (terrible) — pure luck. Taking the best of 10 (green) removes the terrible outcomes. Right: the price is linear extra time.

> 📝 **Version note**
>
> In scikit-learn ≥ 1.4 the default is `n_init='auto'`: 10 runs for `init='random'` but only **1** run for `'k-means++'` (because it is already a smart start). Your grid values `[10, 20, 30]` still work and override this.

LEVEL 3 · ADVANCED — the grid search, step by step

1.8 GridSearchCV for K-Means

This complete template integrates a **Pipeline** for necessary data scaling and uses a **custom evaluation metric**, since GridSearchCV needs a scoring function to compare unsupervised models.



Everything inside ONE Pipeline - so each CV fold repeats the whole chain on its own data

The data flow inside every grid-search trial.

Your template — kmeans_gridsearch.py

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

# 1. Generate synthetic data (replace 'X' with your actual DataFrame)
X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0, random_state=42)

# 2. Define a custom scorer for Unsupervised Learning
# GridSearchCV defaults to 'accuracy' which fails without target labels (y).
```

```python
# We build a custom wrapper to calculate the Silhouette Score instead.
def silhouette_scorer(estimator, X, y=None):
    labels = estimator.predict(X)
    # Silhouette score requires at least 2 clusters to compute mathematically
    if len(set(labels)) < 2:
        return -1
    return silhouette_score(X, labels)

# 3. Build a pipeline
# Scaling is strictly required for distance-based algorithms like K-Means.
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('kmeans', KMeans(random_state=42))
])

# 4. Define the hyperparameter grid
# The 'kmeans__' prefix is required to tell the Pipeline which step to tune.
param_grid = {
    'kmeans__n_clusters': [2, 3, 4, 5, 6, 7],
    'kmeans__init': ['k-means++', 'random'],
    'kmeans__n_init': [10, 20],
    'kmeans__algorithm': ['lloyd', 'elkan']
}

# 5. Configure and run GridSearchCV
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring=silhouette_scorer,
    cv=3,        # Splits data into 3 folds to validate cluster stability
    n_jobs=-1    # Uses all available CPU cores to speed up execution
)

# Fit the grid search to the data (no 'y' variable needed)
print("Running Grid Search...")
grid_search.fit(X)

# 6. Extract results
print(f"Best Silhouette Score: {grid_search.best_score_:.4f}")
print("Best Parameters:", grid_search.best_params_)

# 7. Apply the optimized model
best_model = grid_search.best_estimator_
cluster_assignments = best_model.predict(X)

# Optional: Add the cluster labels back to a pandas DataFrame
# df = pd.DataFrame(X)
# df['Cluster'] = cluster_assignments
```

What every part does

Step	What it does	Why it is needed
1 · `make_blobs`	Creates 500 fake points in 4 blobs.	Practice data. Replace `X` with your real DataFrame.
2 · `silhouette_scorer`	Predicts labels for the held-out fold, then returns the silhouette score (−1 if fewer than 2 clusters).	GridSearchCV's default scorer is accuracy, which fails without target labels `y` . This custom wrapper scores cluster tightness instead.
3 · `Pipeline`	Chains StandardScaler → KMeans.	K-Means is distance-based so scaling is strictly required, and the pipeline keeps scaling inside each CV fold.
4 · `param_grid`	Lists the values to try. Prefix `kmeans__` targets the step named "kmeans".	Without the prefix the Pipeline does not know which step to tune.

Step	What it does	Why it is needed
5 · `GridSearchCV`	Tries all combinations with `cv=3`; `n_jobs=-1` uses all CPU cores.	Automatic testing + speed.
6 · results	`best_score_`, `best_params_`	The winning setting and its score.
7 · `best_estimator_`	The refitted best pipeline; `.predict(X)` gives final labels.	Your final model.

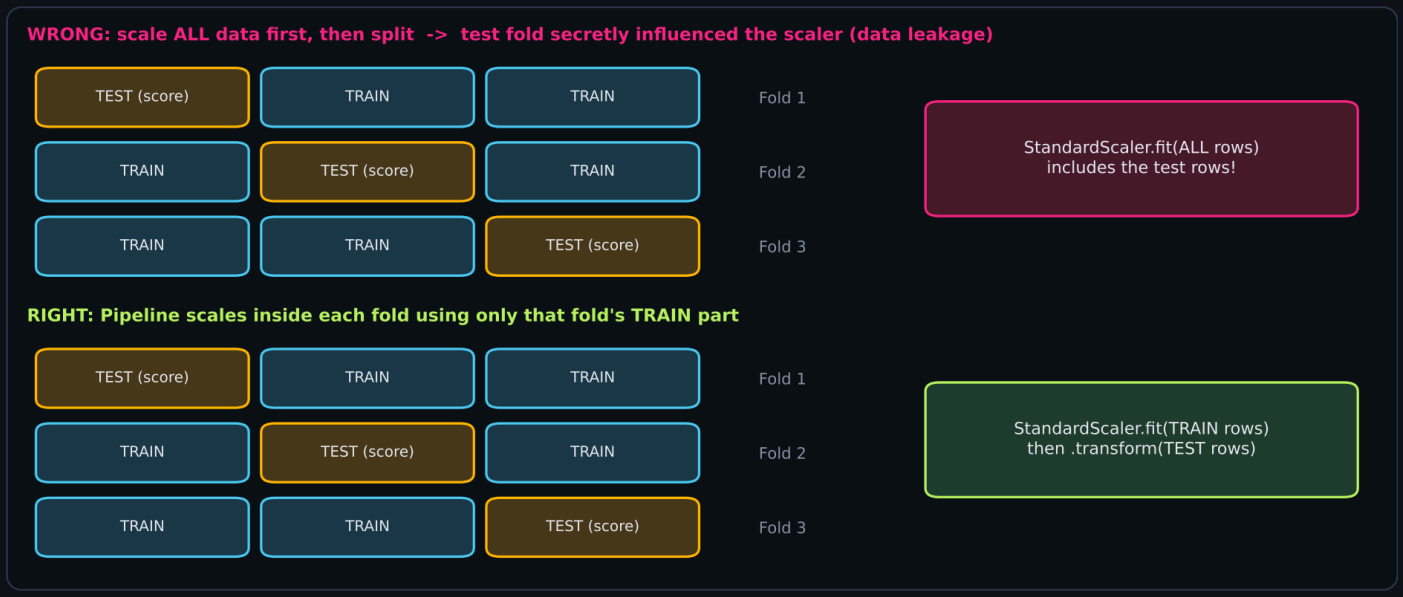
Key implementation details

🔑 Custom Scorer

GridSearchCV is inherently designed for **supervised** learning (x maps to y). By creating `silhouette_scorer`, the grid search ignores the lack of a y variable and evaluates how tightly packed the resulting clusters are.

🔑 Pipeline integration — avoiding data leakage

If you scale your entire dataset *before* splitting it for cross-validation, you cause **“data leakage”**. Wrapping StandardScaler inside the Pipeline ensures the data is scaled correctly within each individual cross-validation fold.



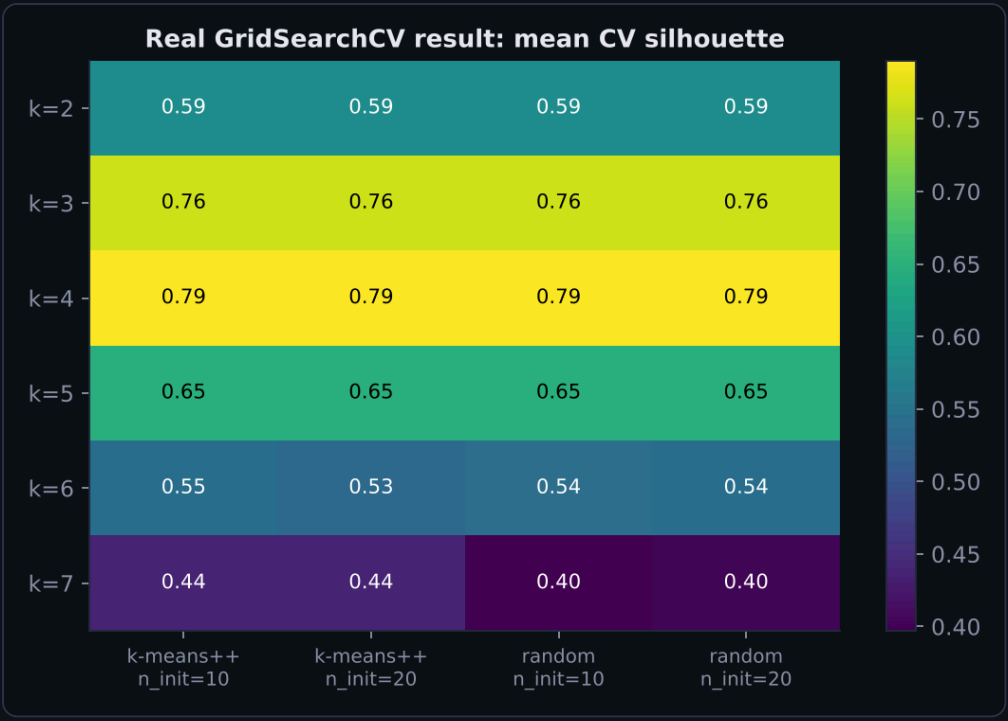
Leakage: if the scaler sees the test fold’s rows, the test fold is no longer truly “unseen”. The Pipeline learns scaling only from each fold’s training part.

🔑 Cross-validation (cv=3)

Even though K-Means is unsupervised, GridSearchCV splits the data into 3 chunks. It trains the centroids on 2 chunks and calculates the Silhouette Score on the 3rd unseen chunk. This shows that your clusters are robust and generalize to new data points, rather than just overfitting the training set.

Real result of running this exact template

We ran it: **48 combinations** × 3 folds in 2.8 s. Best CV silhouette = **0.7903** with `n_clusters=4`, `init=k-means++`, `n_init=10`, `algorithm=lloyd`.



Mean CV silhouette for each k and each (init, n_init) pair (averaged over algorithm). The score depends almost entirely on k; the other settings barely matter on easy, well-separated data.

💡 Why did many settings tie?

On clean, well-separated blobs every sensible setting finds the same clusters, so many rows of `cv_results_` have the identical score 0.7903. GridSearchCV then reports the **first** one it met. When scores tie, prefer the cheaper setting (`n_init=10` , k-means++).

LEVEL 4 · PRO — make K-Means efficient

1.9 Five strategies from Stack Overflow & Reddit data-science communities

These make K-Means computationally efficient *and* practically useful.

① Downsample your data (or use Mini-Batch K-Means)

K-Means is fundamentally calculating averages to find cluster centers; you do not need a million rows to find an accurate average.

⚙️ How it works

Train on a random subset to establish the centroids. Once the centroids are locked in, simply assign the remainder of your data to those centers (`predict`).

✅ How it helps

📉 **How it affects the model**

The impact: Scikit-Learn offers `MiniBatchKMeans`, which processes data in small randomized chunks. It yields virtually identical clusters in a fraction of the time.

② Reduce dimensions with PCA first

K-Means relies on distance calculations (like Euclidean). As you add columns, these distance calculations become heavy and the contrast between distances gets “blurry”.

⚙️ **How it works**

Apply **PCA** before K-Means to compress features into a smaller space (e.g., 50 columns → 5 components).

✅ **How it helps**

📉 **How it affects the model**

The impact: the math runs much faster; removing background “noise” from excessive columns often gives distinctly tighter, more accurate clusters.

③ Switch the underlying algorithm to `"elkan"`

Standard K-Means (Lloyd’s algorithm) is wasteful: it calculates the distance between every data point and every centroid on every iteration.

⚙️ **How it works**

Set `algorithm='elkan'`. It uses a geometry shortcut — the **triangle inequality**. If centroid A is very far from centroid B and a point is very close to A, it provably is far from B, without computing that distance.

✅ **How it helps**

📉 **How it affects the model**

The impact: it skips millions of unnecessary distance calculations, heavily speeding up training on datasets with well-defined clusters. (Trade-off: it keeps extra bounds in memory.)

④ Always scale and normalize your data

K-Means relies entirely on physical distance, so features with larger numbers artificially dominate (see Figure in 0.3).

⚙️ **How it works**

Age (18–80) vs Income ($30,000–$150,000): distances focus entirely on income. Use `StandardScaler` to put every column on the same proportional scale.

✅ **How it helps**

📉 How it affects the model

The impact: the algorithm stops wasting iterations balancing skewed scales, converges faster and produces logically sound clusters.

⑤ Ensure smart initialization

Random initialization can accidentally place starting centroids right next to each other, forcing dozens of iterations to walk them across the data.

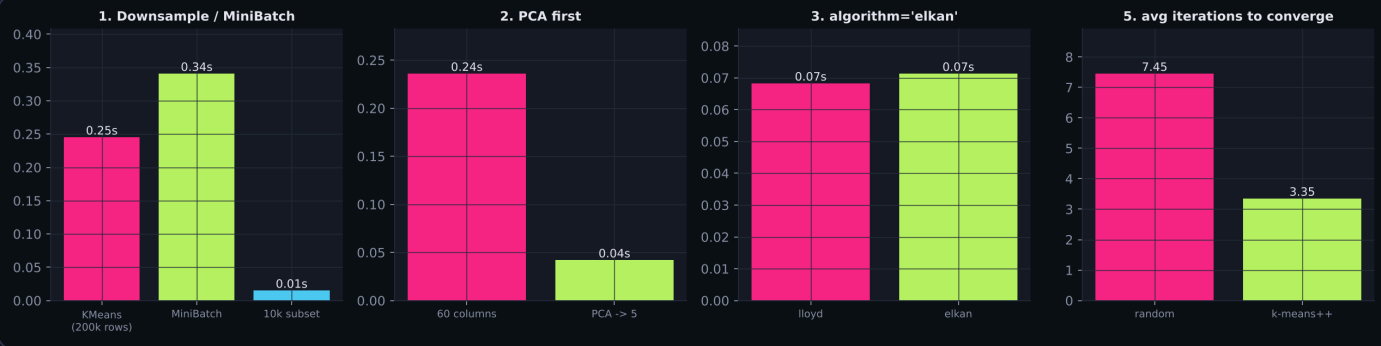
⚙️ How it works

Make sure `init='k-means++'` rather than `'random'`.

✅ How it helps

📉 How it affects the model

The impact: starting points are spaced out intelligently, so far fewer iterations are needed.



Measured on a small 2-CPU sandbox. ① 200k rows: full K-Means 0.25 s, MiniBatch 0.34 s, 10k-row subset 0.01 s. ② 60 columns 0.24 s → after PCA 0.04 s. ③ lloyd 0.07 s vs elkan 0.07 s. ⑤ average iterations: random 7.4 vs k-means++ 3.4.

📝 Honest reading of these numbers

Subsampling (~30× faster here), PCA (~3×) and k-means++ (about half the iterations) clearly help. MiniBatch and elkan gave **no big gain on this small test** — their benefits appear on much larger or better-separated data, so measure on your own data. Quality check: final inertia on all rows was 2,876,431 (full) vs 2,876,571 (MiniBatch) vs 2,878,570 (trained on 10k subset) — nearly identical.

🚀 Pro checklist for K-Means

1. Scale → (optional) PCA → K-Means, all inside a Pipeline.
2. Tune `n_clusters` with silhouette / Calinski-Harabasz, *never* inertia alone.
3. Keep k-means++; keep `n_init` ≥ 10; fix `random_state`.
4. Check the cluster sizes and look at a 2-D plot; K-Means assumes round, similar-size clusters.

PART 2

Hierarchical (Agglomerative) Clustering

LEVEL 1 · BASIC — the idea

2.1 Build a family tree of your data

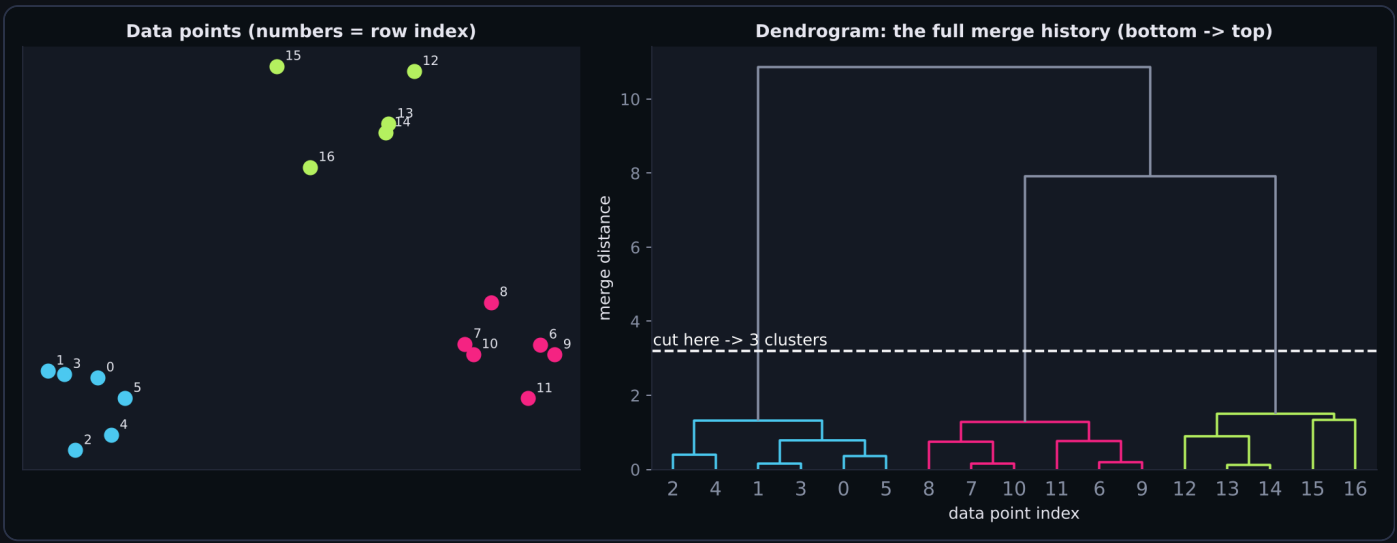
Scikit-Learn's `AgglomerativeClustering` (the standard implementation of hierarchical clustering) builds clusters step by step — either **merging small clusters into larger ones** (agglomerative, bottom-up) or **splitting large ones** (divisive, top-down). Scikit-learn implements the bottom-up version.

> 🍕 **Analogy: friends forming groups at a party**
>
> At the start everyone stands alone. The two people standing closest start talking — that is a group. Then the two closest groups merge, and so on until the whole party is one big circle. If someone records *who joined whom and at what distance*, you get a tree called a **dendrogram**.

1. Every data point starts as its **own cluster**.
2. Find the **two closest clusters** (closeness defined by `linkage` + `metric`).
3. **Merge** them into one.
4. Repeat until only one cluster remains (or until the stopping rule — `n_clusters` or `distance_threshold` — is met).



A toy dataset being merged: 17 singletons → 8 → 4 → 2 clusters.



Left: the points. Right: the dendrogram. Height = distance at which two groups merged. Draw a horizontal cut line: every vertical line it crosses is one cluster.

🔑 Deterministic

Unlike K-Means (which relies on random initialization), hierarchical clustering is **deterministic**: it yields the exact same result every time. There is no `random_state`, no `n_init`. Quality is dictated entirely by how you define "distance": the `linkage` and `metric` parameters.

LEVEL 2 · INTERMEDIATE — hyperparameters

2.2 Complete hyperparameter table

Here is the complete breakdown of the hyperparameters and the preferred values to use when searching for the optimal model.

Hyperparameter	Description	Preferred grid search values
`n_clusters`	The final number of clusters to form.	`[2, 3, 4, 5, 6, 7]` (or `None` if using a threshold)
`linkage`	The strategy used to calculate the distance between two *clusters* to decide if they should merge.	`['ward', 'complete', 'average', 'single']`
`metric`	The mathematical formula used to compute the distance between individual data *points*.	`['euclidean', 'manhattan', 'cosine']`
`distance_threshold`	The distance limit above which clusters will no longer be merged.	`[0.5, 1.0, 5.0, 10.0]` (highly dependent on data scale)

🔑 The core three

The most impactful are `linkage`, `metric` and `n_clusters`. Because the algorithm has no random start, these three **entirely dictate the mathematical logic** of the clustering process.

LEVEL 3 · ADVANCED — deep dive

2.3 `linkage` — cluster-to-cluster distance

⚙️ How it works

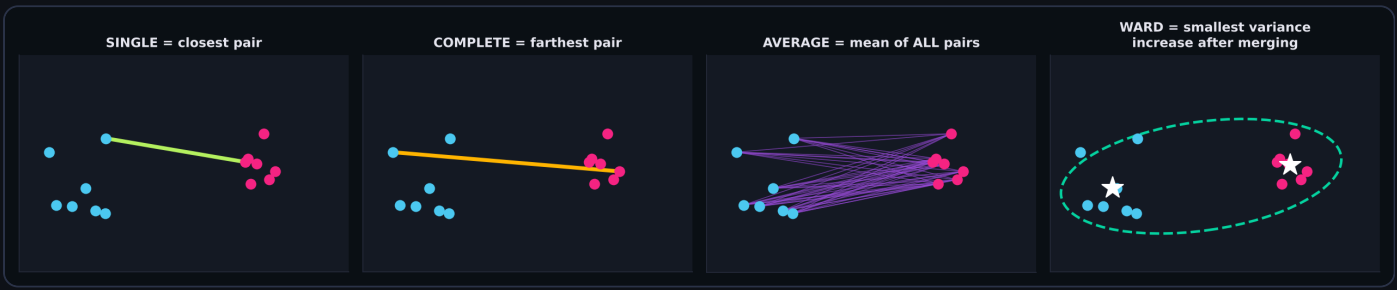
In agglomerative clustering every point starts as its own cluster and the algorithm iteratively merges the two “closest” clusters. `linkage` defines exactly how the distance between two **groups** of points (not single points) is calculated.

✅ How it helps

It dictates the physical shape and structure of the final clusters, so you can adapt the model to whether your natural groupings are dense spheres, long chains or irregularly sized blobs.

📉 How it affects the model

See the four options below.



Four ways to measure “how far apart are these two clusters?”.

linkage	How distance is measured	Effect on the model	Weakness
ward (default)	Variance minimization: merges the two clusters that give the **smallest increase in total within-cluster variance**.	Forces highly spherical, relatively equal-size clusters. Resistant to noise.	Struggles with elongated shapes. **Only works with euclidean.**
complete (maximum)	Distance between the two **furthest** points of the two clusters.	Prevents clusters merging if any extreme points are far apart → very compact, tightly bound clusters; prevents “chaining”.	Highly sensitive to outliers.
average	**Average** distance over all point pairs across the two clusters.	Compromise between single and complete. Handles irregular/unevenly sized clusters better than Ward or Complete; resists chaining.	Medium on everything — rarely the sharpest.
single (minimum)	Distance between the two **closest** points of the two clusters.	Excels at finding non-spherical, wildly-shaped clusters (moons, rings, crescents) by following nearest neighbours.	Extremely sensitive to outliers; **“chaining”** — a bridge of noise points merges two distinct clusters.



Same algorithm, 4 linkages × 3 datasets (silhouette printed on each panel). Ward/average win on round blobs; only single linkage follows the curved moons (but is fragile); ward and single both split the stripes correctly while complete/average cut across them.



Chaining in action: a thin bridge of just 7 noisy points tricks SINGLE linkage into gluing two blobs together (it isolates one stray point instead of splitting the groups); complete and ward are not fooled.

Values for grid search: `['ward', 'complete', 'average', 'single']`

2.4 `metric` — point-to-point distance

⚙️ How it works

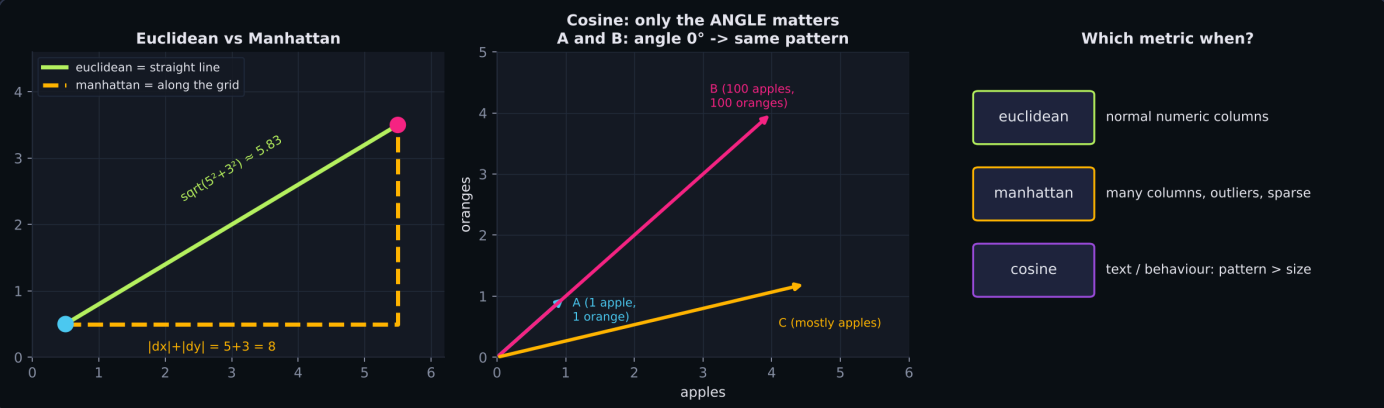
Before linkage can decide which clusters to merge, the algorithm computes a (massive) **distance matrix** measuring the space between every single pair of raw data points. `metric` sets the formula. (In older scikit-learn versions it was called `affinity`.)

✅ How it helps

It determines what “similarity” means for your data. Similarity of text documents needs entirely different math than similarity of GPS coordinates.

📉 How it affects the model

See below.



Left: two ways to travel between the same two points. Middle: with cosine, 1 apple + 1 orange and 100 apples + 100 oranges point the same way (cosine distance 0.0), while Euclidean calls them 140.0 units apart.

metric	How it measures	Best for / effect
euclidean	Standard straight-line geometric distance.	Dense, standard numerical data. Breaks down in high-dimensional spaces (curse of dimensionality) where all points appear equidistant.
manhattan	Moves strictly along a grid: sum of absolute coordinate differences.	Significantly more robust to outliers than Euclidean; performs much better with dozens or hundreds of features, and sparse data.
cosine	Angle between two points rather than their magnitude.	Text clustering and behavioural pattern data. Example: a customer buying 1 apple + 1 orange and another buying 100 + 100 are **identical** by cosine, but **completely different** by Euclidean.

⚠️ Critical constraint

If `linkage='ward'`, the metric **must** be `'euclidean'` — Ward cannot mathematically compute the others. In a grid search, dynamically skip those combinations.

Values for grid search: `['euclidean', 'manhattan', 'cosine']`

2.5 `n_clusters` VS `distance_threshold`

⚙️ How it works

Hierarchical clustering builds a complete tree (dendrogram) from the bottom up, eventually merging everything into one giant cluster. `n_clusters` tells the algorithm exactly **where to slice horizontally** across the tree.

✅ How it helps

It defines the macroscopic granularity of your problem and stops the merging once a business requirement is met (e.g., segment the market into exactly 5 customer profiles).

📉 How it affects the model

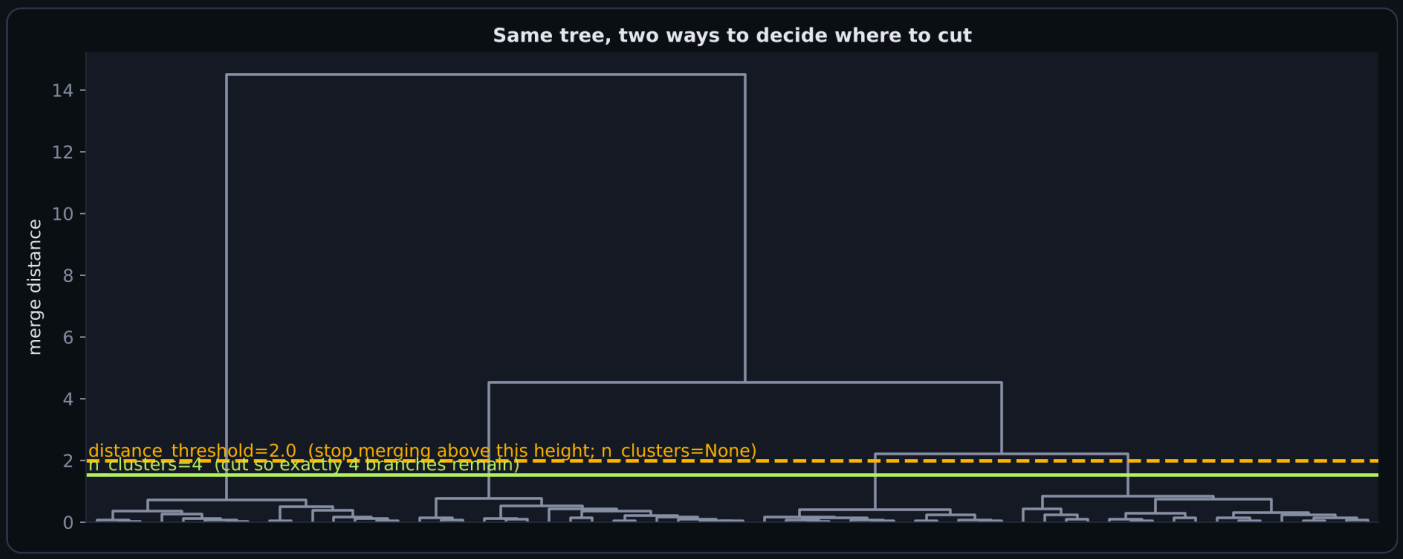
Too low → forced to merge fundamentally different groups just to hit the target (underfitting). Too high → stops too early, leaving fragmented, near-identical micro-clusters (overfitting). Because the algorithm is expensive (**O(n³)** time in worst cases), the right value also avoids wasting memory on unnecessary branches.

🔍 Grid search strategy

Values `[2, 3, 4, 5, 6, 7, 8, 9, 10]`, evaluated against the Silhouette Score.

These two hyperparameters are mutually exclusive — use only one.

Option	How to set	Behaviour
`n_clusters`	`n_clusters=4` (leave `distance_threshold=None`)	You tell the algorithm exactly how many groups to return. It keeps merging until it hits that number, *regardless of how far apart they are*.
`distance_threshold`	`n_clusters=None, distance_threshold=5.0`	Merges bottom-up and **stops once the distance between the closest remaining clusters exceeds your threshold**. The algorithm decides the number of clusters from a spatial cutoff.



The same dendrogram cut two ways. A threshold of 2.0 happened to produce 4 clusters here. Tip: look for the tallest vertical gaps — cutting inside a big gap gives the most natural clusters.

2.6 A crucial warning for grid searching

> ⚠️ **You cannot easily use GridSearchCV with hierarchical clustering**
>
> GridSearchCV relies on cross-validation: it splits data into train and test sets and uses `.predict()` to assign test data to clusters. **Hierarchical clustering has no** `.predict()`. It is **transductive**: it only groups the exact data it was fitted on and cannot assign new, unseen points.
>
> To tune, write a custom `for` loop that goes through the parameter combinations, fits on the **entire dataset** and calculates a metric like the Silhouette Score for each.

2.7 Grid-search template for hierarchical clustering

Use Scikit-Learn's `ParameterGrid`: it generates exactly the same combinations as GridSearchCV, so you can loop through them and score the entire dataset.

Your template — hierarchical_gridsearch.py

```
import pandas as pd
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_moons

# 1. Generate synthetic data
X, _ = make_moons(n_samples=500, noise=0.1, random_state=42)

# 2. Scale the data (Mandatory for distance-based clustering)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Define the hyperparameter grid
# Note: If 'ward' linkage is tested, metric MUST be 'euclidean'
param_grid = {
    'n_clusters': [2, 3, 4, 5, 6],
    'linkage': ['ward', 'complete', 'average', 'single'],
    'metric': ['euclidean', 'manhattan']
}

# 4. Initialize tracking variables
best_score = -1.0
best_params = None
best_labels = None

# 5. Execute the Grid Search manually using ParameterGrid
print("Running Grid Search...")
for params in ParameterGrid(param_grid):

    # Catch the mathematical incompatibility between Ward and non-Euclidean metrics
    if params['linkage'] == 'ward' and params['metric'] != 'euclidean':
        continue

    try:
        # Initialize and fit the model
        model = AgglomerativeClustering(**params)
        labels = model.fit_predict(X_scaled)

        # Calculate the score (requires at least 2 clusters and less than N clusters)
        if 1 < len(set(labels)) < len(X_scaled):
            score = silhouette_score(X_scaled, labels)

            # Update best score
            if score > best_score:
                best_score = score
                best_params = params
```

```
                best_labels = labels

    except Exception as e:
        # Catch any other incompatible parameter combinations
        pass

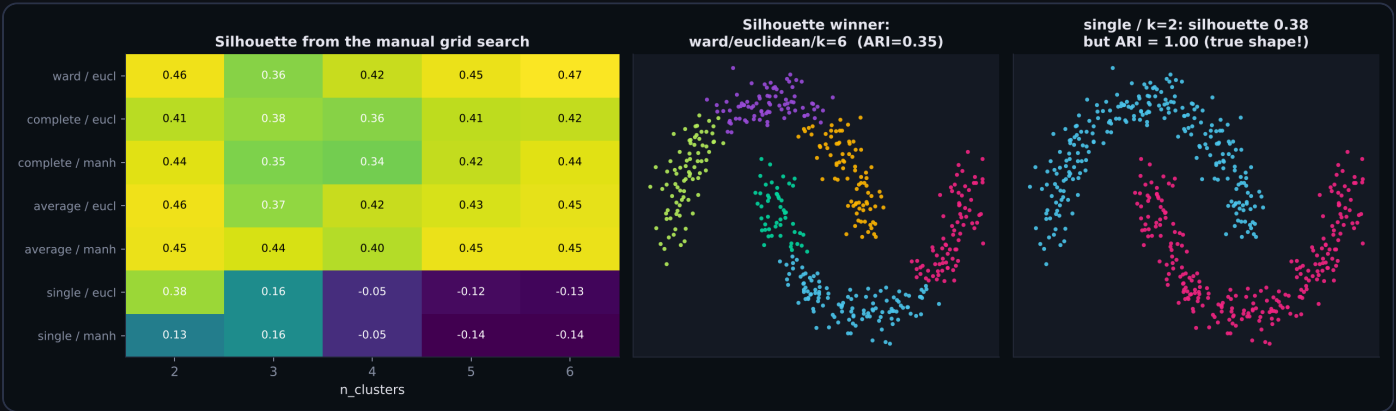
# 6. Output the optimal configuration
print(f"Best Silhouette Score: {best_score:.4f}")
print(f"Optimal Hyperparameters: {best_params}")

# Optional: Attach optimal labels back to the dataset
# df = pd.DataFrame(X)
```

Part	Meaning
`make_moons` + `StandardScaler`	Two interleaving crescents as test data, then scaling (mandatory for distance-based clustering).
`param_grid`	5 values of k × 4 linkages × 2 metrics = 40 combos. Ward + manhattan is invalid, so 5 are skipped.
`continue` on ward ≠ euclidean	Skips the mathematically impossible combination.
`try / except`	Catches any other incompatible combination so the loop never crashes.
`1 < len(set(labels)) < len(X)`	Silhouette needs at least 2 clusters and fewer clusters than points.
best_score / best_params / best_labels	Keeps the winner so far; at the end you print it and can attach labels back to a DataFrame.

Real result — and an important lesson

Running your template on the moons data gives: best silhouette **0.4685** with `linkage=ward`, `metric=euclidean`, `n_clusters=6`.



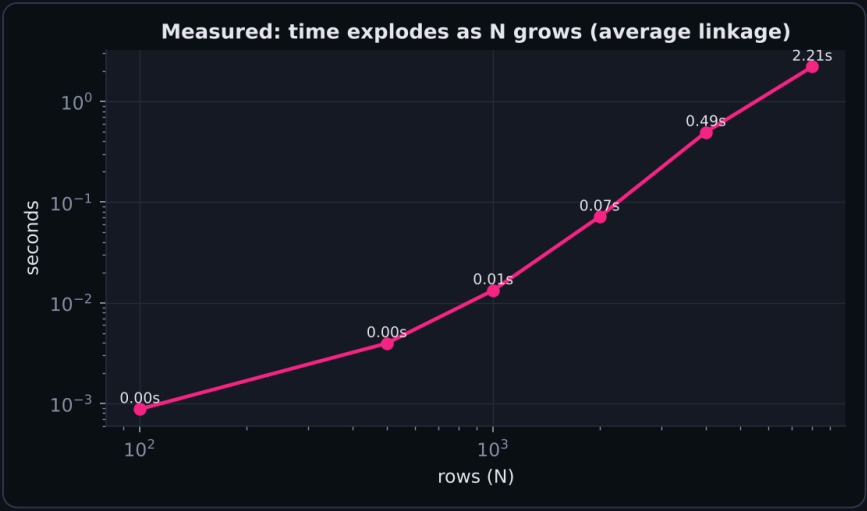
Left: all silhouette scores. Middle: the silhouette "winner" chops the moons into 6 round pieces (ARI vs true moons = 0.35). Right: single linkage with k=2 recovers the true moons perfectly (ARI = 1.0) yet has a *lower* silhouette.

> **🚀 Lesson: a score is a guide, not a judge**
>
> Silhouette prefers compact round blobs, so it cannot reward a correct curved shape. If you know your data has curved/elongated groups, **look at the plots**, use a density-based method (DBSCAN) or a metric that suits the shape. And if you do have true labels for a benchmark, compare with ARI.

LEVEL 4 · PRO — make hierarchical clustering efficient

2.8 Four strategies from data-science communities

Hierarchical clustering is notoriously **memory-hungry and computationally expensive** (O(N³) time in the worst case; it stores an N × N distance matrix).



Measured: 2,000 rows → 0.07 s, 4,000 → 0.49 s, 8,000 → 2.21 s. Doubling the data costs ~6× the time (and 4× the memory).

① Impose connectivity constraints (the KNN-graph shortcut)

By default the algorithm calculates distance between every pair of clusters before each merge. **How it works:** pass a `connectivity` matrix (usually built from a K-Nearest-Neighbors graph). **Impact:** clusters can only merge if they are physically adjacent, skipping millions of checks for clusters on opposite sides of the data.

```
from sklearn.neighbors import kneighbors_graph
conn = kneighbors_graph(X, n_neighbors=10, include_self=False)
model = AgglomerativeClustering(n_clusters=6, linkage='ward', connectivity=conn)
```

② Precompute the distance matrix

During a grid search over linkage / n_clusters, the physical distance between raw points never changes. **How it works:** calculate the matrix once with `scipy.spatial.distance`, pass it in and set `metric='precomputed'`. **Impact:** the heaviest work is skipped on every later run.

```
from scipy.spatial.distance import pdist, squareform
D = squareform(pdist(X_scaled))          # computed ONCE
model = AgglomerativeClustering(n_clusters=4, linkage='average', metric='precomputed').fit(D)
```

> ⚠️ **Ward + precomputed**
>
> `linkage='ward'` cannot be combined with `metric='precomputed'` (Ward needs the raw coordinates). Use complete / average / single.

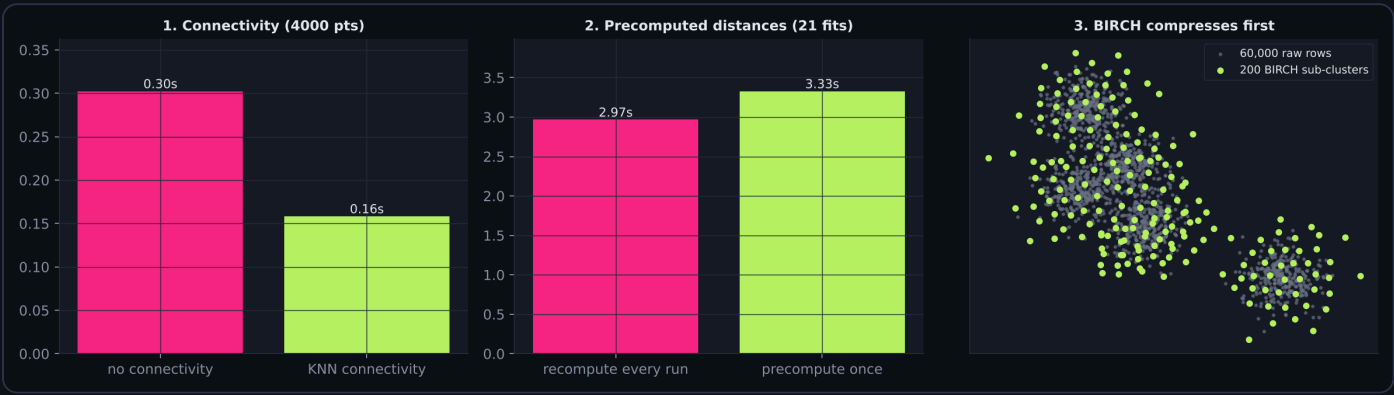
③ Use BIRCH for datasets over 10,000 rows

Standard hierarchical clustering often crashes RAM beyond 10,000–20,000 rows because of the N × N matrix. **How it works:** use `Birch` as a pre-processing step: it scans the huge dataset and compresses it into a smaller set of “sub-clusters” that preserve the spatial structure. Then run hierarchical clustering on the sub-clusters instead of raw data. **Impact:** millions of rows become feasible.

```
from sklearn.cluster import Birch
b = Birch(n_clusters=None, threshold=0.5).fit(X_big)
sub = b.subcluster_centers_                       # far fewer points
labels_sub = AgglomerativeClustering(n_clusters=5).fit_predict(sub)
```

④ Flatten the data with PCA

Distance calculations get heavier with every column. **How it works:** apply PCA (e.g., 100 columns → 10) before clustering. **Impact:** faster execution, and removing low-importance “noise” columns prevents the chaining effect where distinct clusters merge due to outlier dimensions.



Measured: ① connectivity 0.3 s → 0.16 s (4,000-point swiss roll). ② 21 grid fits: 2.97 s → 3.33 s including the one-time matrix. ③ 60,000 rows compressed to 200 BIRCH sub-clusters in 1.14 s.

📝 Honest reading

On this small test the connectivity and precompute gains are modest (~1.5× and ~1.05×). They grow with data size. Note too that adding connectivity changes the clusters (agreement with the plain run, ARI = 0.45) — it is a modelling choice (“only neighbours may merge”), not a free speed-up. BIRCH is the big lever for large N.

PART 3

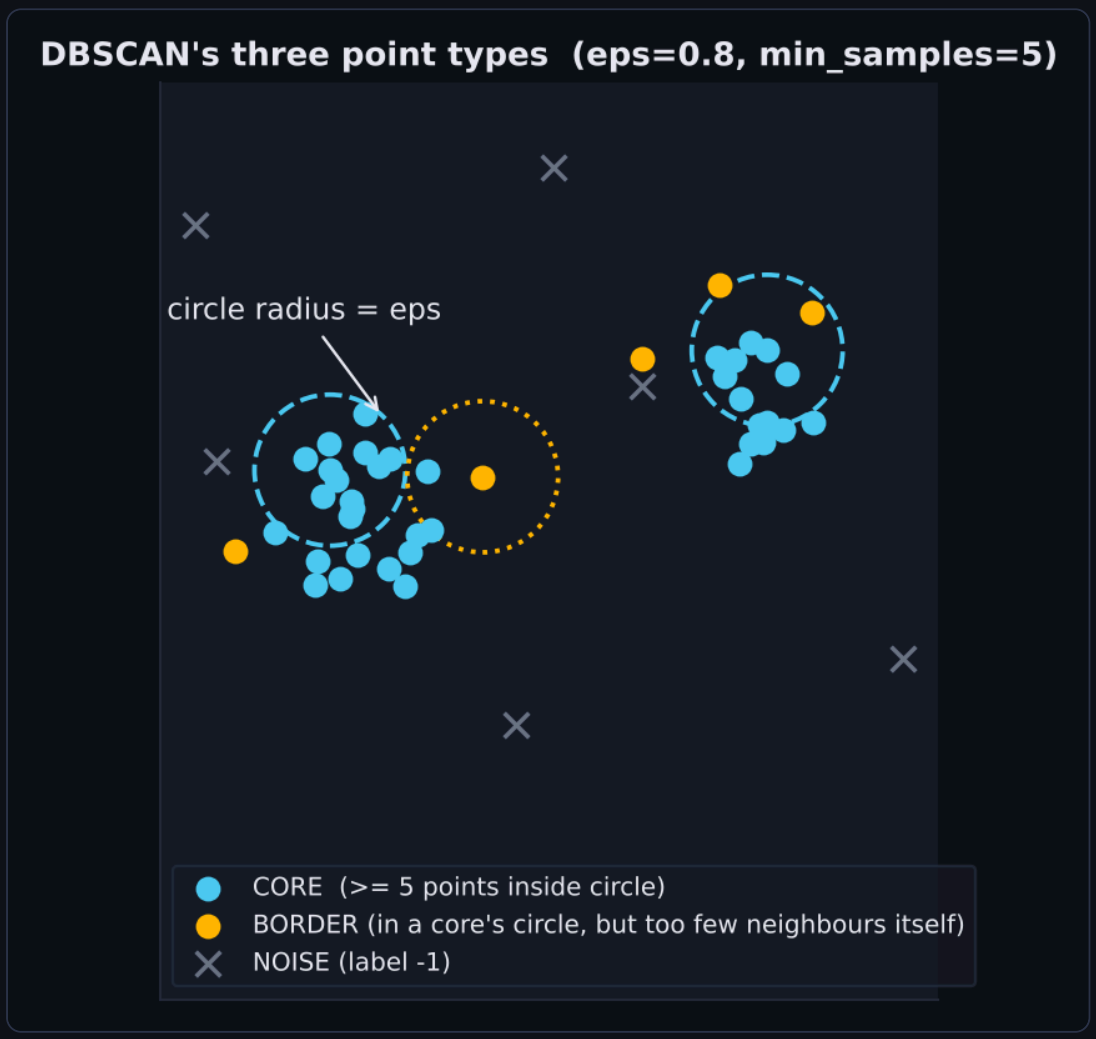
DBSCAN Clustering

LEVEL 1 · BASIC — the idea

3.1 Clusters are crowded neighbourhoods

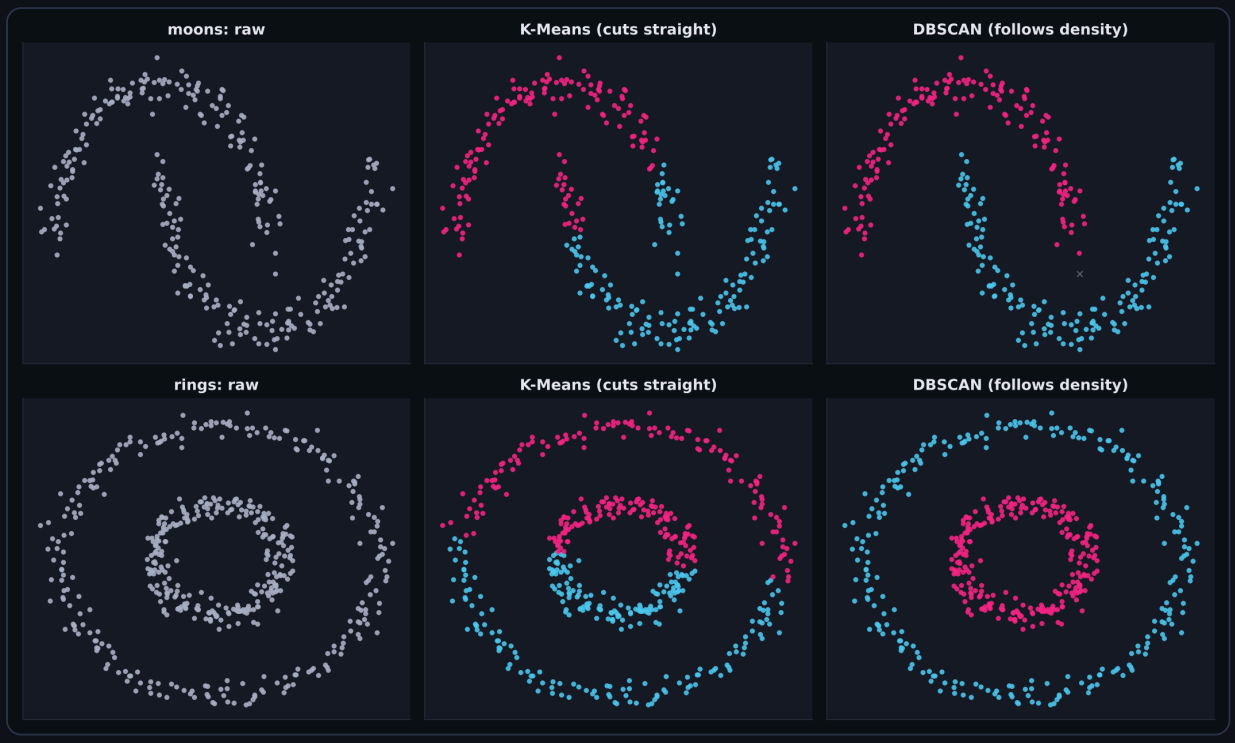
DBSCAN (Density-Based Spatial Clustering of Applications with Noise) relies on finding **dense regions** of data and separating them from **sparse regions**. Unlike K-Means, it does **not require the number of clusters in advance**, but its performance is highly sensitive to its hyperparameters.

> 🍕 **Analogy: islands in the sea**
>
> Look at a map at night: cities are bright clusters of lights, farms are isolated dots. A “city” is a place where at least *N* houses lie within walking distance of each other. Isolated houses are not part of any city — they are **noise**. DBSCAN finds cities of any shape (long like a coastline, curved like a crescent) and flags lonely houses as outliers.



The three kinds of points. Draw a circle of radius eps around a point and count the points inside: ≥ min_samples → CORE (it can grow a cluster). Inside a core’s circle but with too few neighbours itself → BORDER (joins the cluster, cannot expand it). Neither → NOISE, label −1.

1. Pick an unvisited point; count neighbours inside its `eps` circle.
2. If count ≥ `min_samples` → it is a **core point**: start a new cluster.
3. Every neighbour joins; each neighbour that is itself core spreads the cluster further (chain reaction).
4. When no more points can be reached, the cluster is finished. Points never reached = **noise (−1)**.



Why DBSCAN exists: K-Means draws straight boundaries and cuts curved shapes in half; DBSCAN follows the density and gets moons and rings right.

LEVEL 2 · INTERMEDIATE — hyperparameters

3.2 Complete hyperparameter table

Hyperparameter	Description	Preferred grid search values
`eps`	The maximum distance (radius) between two points for them to be considered neighbors.	`[0.1, 0.5, 1.0, 5.0]` (dataset-scale dependent)
`min_samples`	The minimum number of points required within the eps radius to form a “core point” (a dense region).	`[3, 5, 10, 15, 20]`
`metric`	The mathematical formula used to compute the distance between points.	`['euclidean', 'manhattan', 'cosine']`
`algorithm`	The underlying search algorithm used to find nearest neighbors (`'auto'` , `'ball_tree'` , `'kd_tree'` , `'brute'`).	`['auto']` (rarely needs tuning)

Hyperparameter	Description	Preferred grid search values
`leaf_size`	The leaf size passed to BallTree / KDTree to balance build time vs. query time.	`[30]` (leave as default)
`p`	The power parameter for the Minkowski metric (if `metric='minkowski'`).	`[1, 2]` (1 is Manhattan, 2 is Euclidean)
`metric_params`	Additional keyword arguments passed to the chosen metric function.	`None` (leave as default)
`n_jobs`	The number of CPU cores to use for parallelizing the neighbor search.	`-1` (uses all cores; not a tuning parameter)

🔑 The big picture

DBSCAN's success relies almost entirely on balancing **two** hyperparameters: `eps` and `min_samples`. Together they define exactly what constitutes a "dense" cluster for *your* dataset. Metric is a possible third. The other five are defaults / engineering knobs.

LEVEL 3 · ADVANCED — deep dive

3.3 `eps` (epsilon) — the neighbourhood radius

⚙️ How it works

Imagine placing a compass on a data point and drawing a circle (a sphere in 3-D or more) of radius `eps`. The algorithm counts how many other points fall inside.

✅ How it helps

It sets the **spatial scale** of your problem: how close two points must be to belong to the same group.

📉 How it affects the model

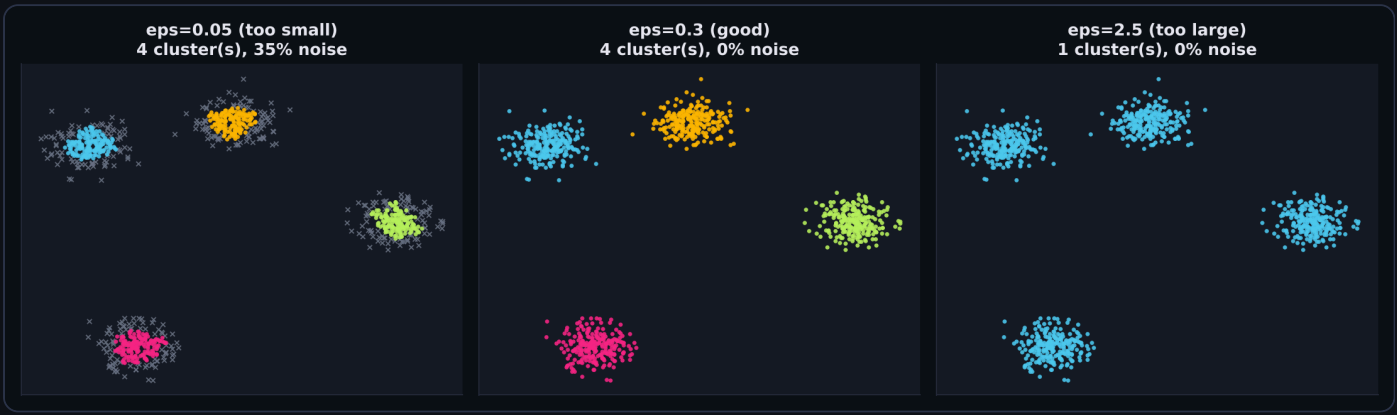
Too small: the circles can't catch neighbours; almost every point becomes noise (label −1) → zero clusters.
Too large: circles bridge the gaps between separate groups; everything merges into one giant cluster.
Cascading effect: clusters grow organically — a point inside a core's eps joins, then *its* circle is checked to grow further. A slightly-too-large eps causes **"chaining"**: the cluster bleeds across a thin bridge into another group. So a small change in eps can drastically change the model.

🔍 Grid search strategy

eps is extremely sensitive to data scale (if data ranges 0–1, eps=5.0 is useless), so **always StandardScaler first**. After scaling, try `[0.1, 0.3, 0.5, 0.7, 1.0]` — or the wider `[0.1, 0.3, 0.5, 0.8, 1.0, 1.5]`.



eps on the two-moons data: too small = fragments and noise, just right = 2 moons, too large = one blob.



eps on 4 blobs: 0.05 shatters everything, 0.3 finds 4 clusters, 2.5 merges them all into one.

3.4 `min_samples` — the density threshold

⚙️ How it works

After the eps circle is drawn, the algorithm counts the points inside. `min_samples` is the strict quota: if the count meets or exceeds it, the centre is crowned a **core point**, which gives it the power to create or expand a cluster.

✅ How it helps

It is the algorithm's **noise filter** and solidity requirement: random sparse scatterings are not mistaken for meaningful clusters.

📉 How it affects the model

High values: only extremely dense, tightly-packed clusters survive; reliable, but large parts of the data are discarded as noise and looser valid groupings may be missed.
Low values: sparse, loose points can form clusters, but noise is captured as valid groups. A value of 1 turns DBSCAN into a simple nearest-neighbour linker; 2 or 3 makes small meaningless clusters from random noise.

🔍 Grid search strategy

Rule of thumb: **min_samples = 2 × D** (D = number of columns). A 5-column dataset → baseline 10. Grid-search a bracket around it, e.g. `[5, 8, 10, 12, 15]` (the table lists `[3, 5, 10, 15, 20]`).



Fixed eps, growing min_samples on 4 blobs plus scattered noise: min_samples=2 lets random specks form 15 tiny clusters; 12 finds the 4 real ones; 60 still finds 4 but is stricter, so the noise share rises (4% → 6% → 7%) as cluster edges are trimmed.

3.5 `metric`

⚙️ How it works

Defines how distance is measured when drawing the eps circle.

✅ How it helps

Adjusts the definition of “closeness” to the nature of your data.

📉 How it affects the model

Euclidean (the default) works well for standard continuous variables, but in high dimensions Euclidean distances become distorted. Switching to **manhattan** (grid distance) or **cosine** (angle) can completely change which points fall inside the eps radius, often giving cleaner clusters for complex data.

🔍 Grid search strategy

`['euclidean', 'manhattan', 'cosine']`. (Reuse the metric pictures from Part 2.)

3.6 How eps and min_samples interact: Density = Mass ÷ Volume

Tuning DBSCAN means understanding that the two are two sides of the same equation:

Density = Mass / Volume

- `min_samples` controls the **Mass** (how many points).
- `eps` controls the **Volume** (how big the circle is).

If you **increase eps** (volume) you generally need to **increase min_samples** (mass) to keep the same strictness. Tuning one without the other swings the model rapidly from “zero clusters” to “everything in one”.

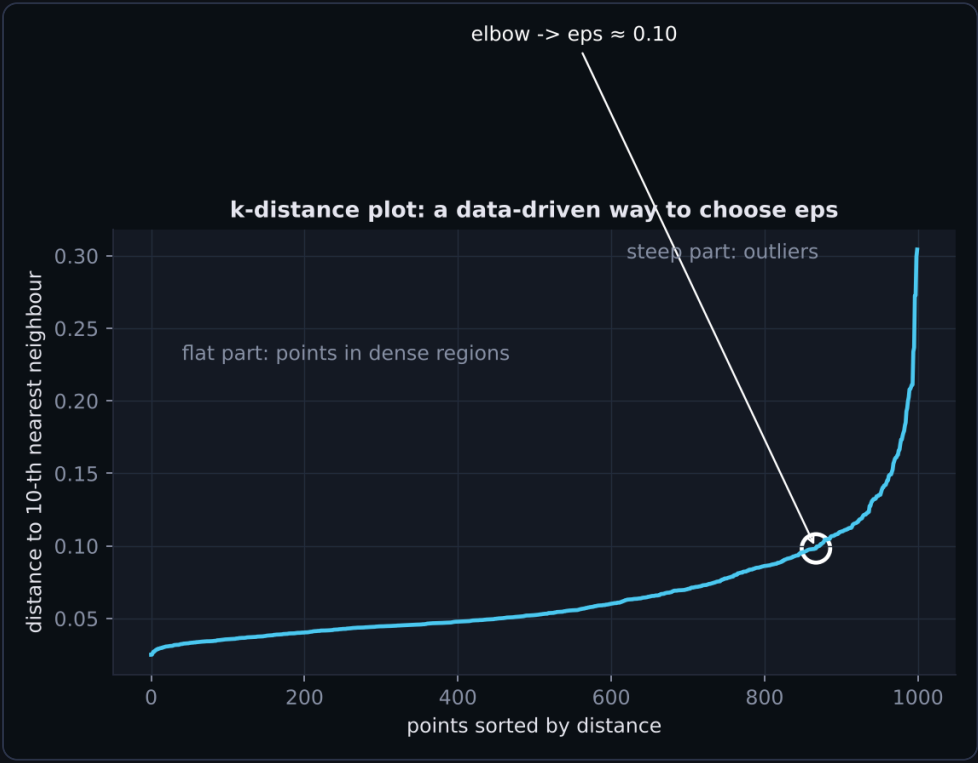
🍕 Analogy: a crowded room

“Crowded” = at least 10 people within arm’s reach. If you double the arm length (eps), you will find more people nearby, so to keep the same definition of “crowded” you must raise the required headcount too.

3.7 Choosing eps with the k-distance plot (pro trick)

Instead of guessing eps: for every point compute the distance to its *k*-th nearest neighbour (k = min_samples), sort, and plot. The curve is flat where points are in dense regions and shoots up for outliers. The **elbow** is a data-driven starting value for eps.

```
from sklearn.neighbors import NearestNeighbors
k = 10                                  # = min_samples
d, _ = NearestNeighbors(n_neighbors=k).fit(X_scaled).kneighbors(X_scaled)
plt.plot(np.sort(d[:, -1]))             # look for the elbow
```



k-distance plot for the 4-blob data (k=10): elbow near eps ≈ 0.098. Use it as the centre of your grid, not as the final answer — the grid search below found 0.3 scored better.

3.8 Grid searching DBSCAN

⚠️ Same rule as hierarchical

DBSCAN is **transductive**: no `.predict()` for new data. Scikit-Learn’s native GridSearchCV will crash. You must loop through `ParameterGrid`, train, and evaluate the resulting labels with e.g. the Silhouette Score. Your loop must also **ignore combinations that label the entire dataset as noise** (or produce < 2 clusters), because silhouette throws an error then. Because outliers get label −1, your scoring logic must ignore noise and ensure at least two distinct valid clusters exist before scoring.

Your template — dbscan_gridsearch.py

```python
import numpy as np
import pandas as pd
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

# 1. Generate synthetic data
X, _ = make_blobs(n_samples=1000, centers=4, cluster_std=0.8, random_state=42)

# 2. Scale the data (Mandatory for eps to function logically)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Define the hyperparameter grid
param_grid = {
    'eps': [0.1, 0.3, 0.5, 0.7, 1.0, 1.5],
    'min_samples': [3, 5, 10, 15, 20]
}

# 4. Initialize tracking variables
best_score = -1.0
best_params = None
best_labels = None

# 5. Execute the Grid Search manually
print("Running Grid Search for DBSCAN...")
for params in ParameterGrid(param_grid):

    # Initialize and fit the model (using all CPU cores)
    model = DBSCAN(**params, n_jobs=-1)
    labels = model.fit_predict(X_scaled)

    # Identify how many actual clusters were found, ignoring noise (-1)
    unique_labels = set(labels)
    n_clusters = len(unique_labels) - (1 if -1 in labels else 0)

    # Silhouette score mathematically requires at least 2 clusters
    if n_clusters >= 2:
        # Calculate score using only the non-noise points
        # Optional: You can penalize the score if too much data is classified as noise
        score = silhouette_score(X_scaled, labels)

        if score > best_score:
            best_score = score
            best_params = params
            best_labels = labels

# 6. Output the optimal configuration
if best_params is not None:
    print(f"Best Silhouette Score: {best_score:.4f}")
    print(f"Optimal Hyperparameters: {best_params}")

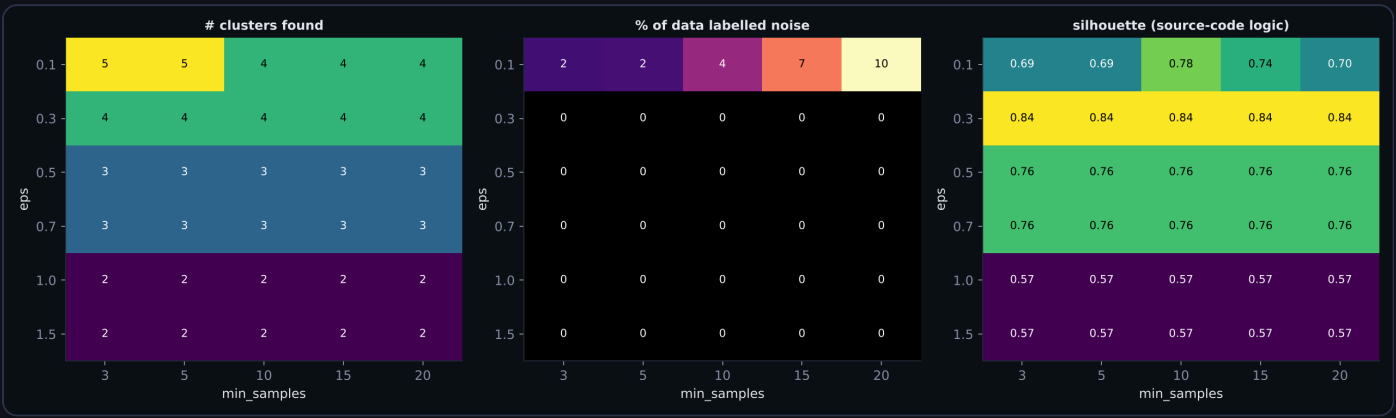
    # Calculate noise percentage for the best model
    noise_ratio = list(best_labels).count(-1) / len(best_labels) * 100
    print(f"Percentage of data classified as noise: {noise_ratio:.2f}%")
else:
    print("No parameter combination found at least 2 distinct clusters.")
```

Part	Meaning
`make_blobs(1000, 4 centres)` + scaler	Test data and mandatory scaling so eps behaves logically.
`param_grid`	6 eps values × 5 min_samples = 30 combinations.
`DBSCAN(**params, n_jobs=-1)`	Fit with every CPU core (speeds the neighbour search, does not change clusters).

Part	Meaning
`n_clusters = len(unique) - (1 if -1 in labels)`	Counts real clusters, not the noise label.
`if n_clusters ≥ 2`	Silhouette needs ≥ 2 clusters.
noise ratio print	Tells you what % of data was thrown away — always report it.

Real result

Your template picks `eps=0.3`, `min_samples=3`: silhouette **0.8392**, **0.0% noise**. Look at the three heatmaps — they show how eps and min_samples interact.



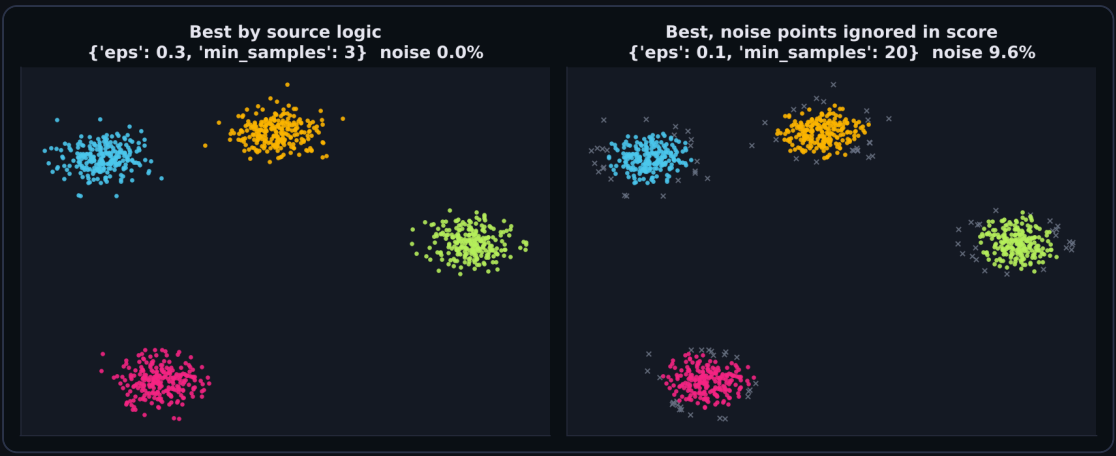
Left: number of clusters found. Middle: % noise. Right: silhouette. Rows of eps dominate: 0.1 loses data to noise and over-splits; 0.3 finds exactly 4; ≥ 1.0 merges blobs into 2. min_samples barely matters on this clean data.

> **⚠ A subtle issue in the template**
>
> The code comment says “calculate score using only the non-noise points”, but `silhouette_score(X_scaled, labels)` actually scores *all* points, treating noise (−1) as one big cluster. A fairer approach: score only non-noise points **and** report the noise % (or penalize it), so a setting can’t look great by discarding half the data.

Improved scoring inside the loop

```
mask = labels != -1
if len(set(labels[mask])) >= 2:
    score = silhouette_score(X_scaled[mask], labels[mask])
    noise_pct = 100 * (1 - mask.mean())
    # optional: score_adj = score * (1 - noise_pct/100)   # penalise discarded data
```

With non-noise-only scoring the winner becomes `eps=0.1, min_samples=20` (silhouette 0.8583) — but it throws away **9.6% of the data** as noise. A higher score bought by deleting points is a warning sign, not a win.



Left: the template's winner keeps every point and finds the 4 blobs. Right: the "better" non-noise score comes from discarding ~10% of points.

LEVEL 4 · PRO — make DBSCAN efficient

3.9 Four strategies from data-science communities

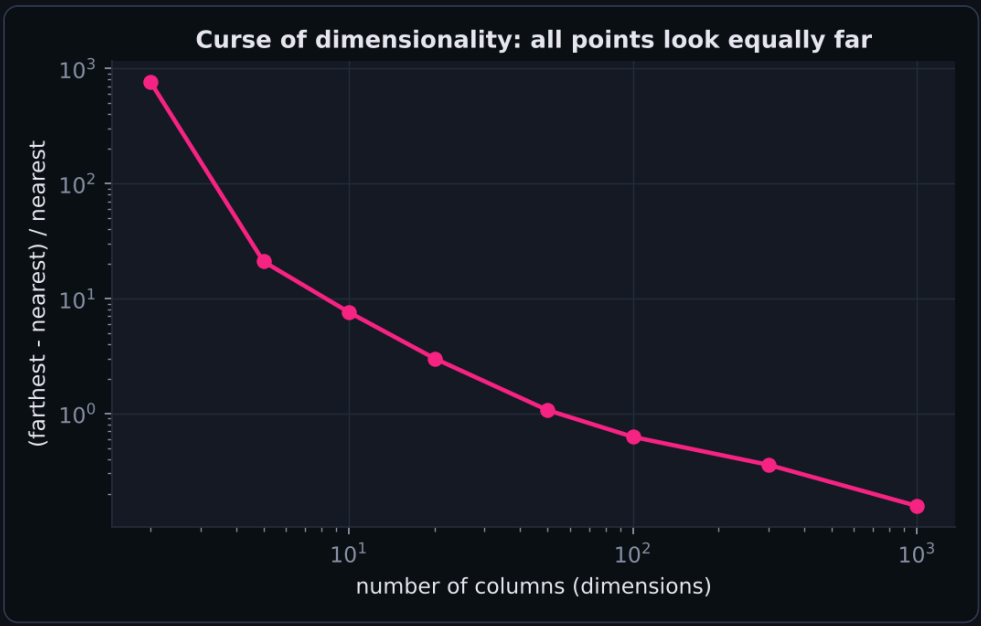
DBSCAN has heavy memory and distance-calculation requirements.

① Maximize CPU cores with `n_jobs=-1`

By default scikit-learn runs on a single core and DBSCAN measures many point-to-point distances. **How:** `n_jobs=-1` splits the work across all cores. **Impact:** big time cut on large datasets with **zero impact** on the final clusters.

② Reduce dimensionality first (PCA or UMAP)

High-dimensional data (dozens of columns) triggers the curse of dimensionality: points become mathematically equidistant and distances eat RAM. **How:** compress to **2–5** representative columns with PCA or UMAP before DBSCAN. **Impact:** faster, and density is identified far more accurately in low dimensions.



Measured on random points: with 2 columns the farthest pair is 765.01× farther than the nearest pair; with 1,000 columns only 0.16×. When “near” and “far” are almost the same, eps loses meaning.

③ Optimize the nearest-neighbour tree

DBSCAN uses a backend search structure to find points inside the eps circle instead of testing every pair. `algorithm` defaults to `'auto'`. **How:** for low-dimensional data (< 20 columns) set `algorithm='kd_tree'`; for higher dimensionality use `'ball_tree'` so the KD-tree does not become a bottleneck. **Impact:** forces the most efficient spatial index for your data shape.

④ The ultimate Reddit tip: switch to HDBSCAN

Standard DBSCAN struggles badly when clusters have **varying densities** (one tight, one loose). **How:** HDBSCAN (Hierarchical DBSCAN) drops `eps` entirely and needs mainly `min_samples` (and `min_cluster_size`); it extracts clusters at many density levels at once. **Impact:** no more eps tuning, and varying densities are handled.



Left: one loose and two tight clusters. Middle: one eps cannot suit both — the loose cluster is eroded into noise (×). Right: HDBSCAN finds all three with no eps. (Built into scikit-learn ≥ 1.3 as `sklearn.cluster.HDBSCAN`.)



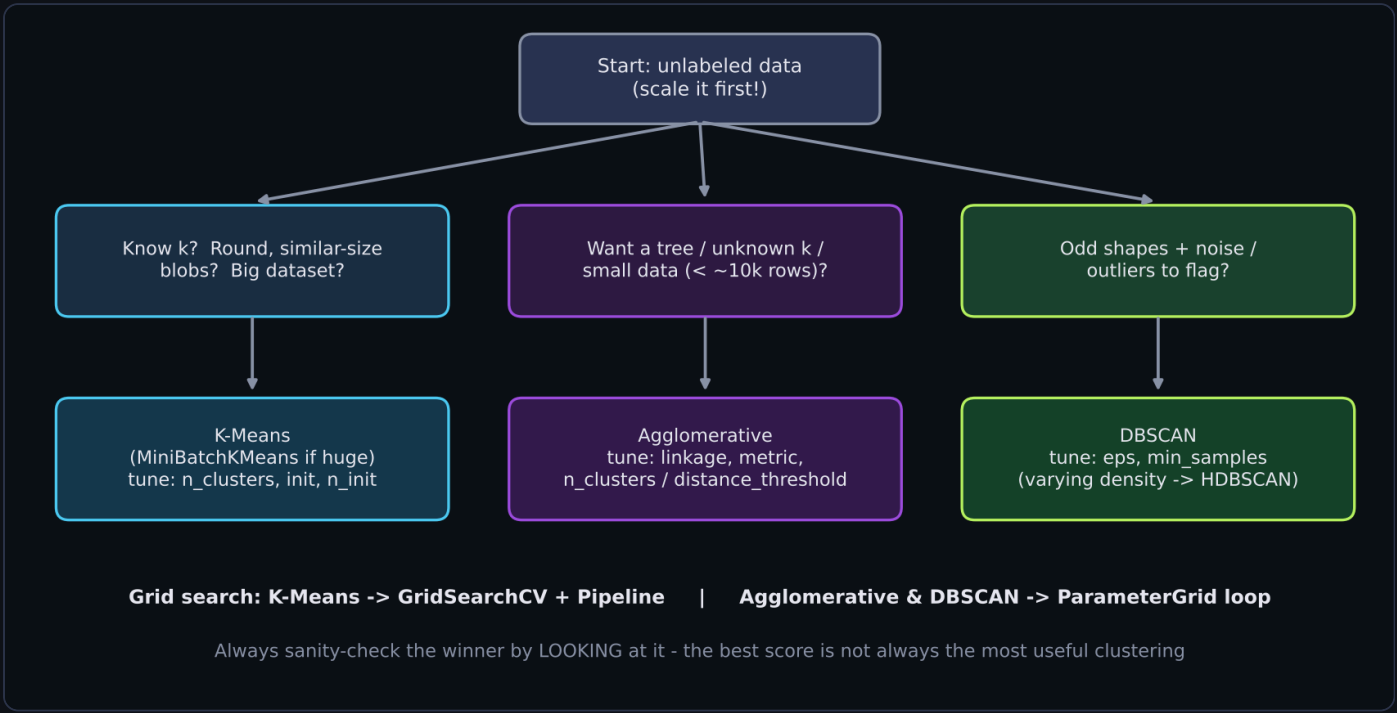
Measured on a 2-CPU sandbox. ① 40,000 rows: 1.9 s → 1.14 s with n_jobs=-1. ② 80 columns 0.22 s vs PCA→3 0.23 s. ③ brute 0.33 s vs kd_tree 0.23 s on 15k rows.

> **📝 Honest reading**
>
> n_jobs shows a clear gain, and kd_tree beats brute slightly. PCA didn't speed this small 8,000-row example (the neighbour search was already cheap) — its main DBSCAN benefit is *accuracy* of density in high dimensions, and RAM savings at scale.

PART 4

Pro Playbook

4.1 Which algorithm should I use?



A quick decision map.

4.2 Side-by-side cheat sheet

	K-Means	Agglomerative	DBSCAN
Must choose k?	Yes (`n_clusters`)	Yes, or use `distance_threshold`	**No**
Cluster shape	Round, similar size	Depends on linkage (single = any shape)	**Any shape**
Outliers	Pulled into clusters	Sensitive (complete/single)	**Labelled −1**
Random?	Yes (init) → n_init, random_state	**Deterministic**	Deterministic (border-point order can vary slightly)
Key hyperparameters	`n_clusters, init, n_init`	`linkage, metric, n_clusters`	`eps, min_samples` (+ metric)
Has `.predict()` ?	**Yes**	No	No
Grid search tool	`GridSearchCV` + Pipeline + custom scorer	`ParameterGrid` loop	`ParameterGrid` loop
Speed on big data	**Fast** (O(k·n·T))	Slow (O(n²)–O(n³)) memory N×N	Medium; depends on neighbour search

	K-Means	Agglomerative	DBSCAN
Main speed-ups	MiniBatch, PCA, elkan, scaling, k-means++	connectivity, precompute, BIRCH, PCA	n_jobs=-1, PCA/UMAP, kd/ball tree, HDBSCAN
Scale data first?	**Yes**	**Yes**	**Yes** (eps depends on it)

4.3 Typical grid-search ranges at a glance

Algorithm	Grid
K-Means	`n_clusters [2–10]` · `init ['k-means++','random']` · `n_init [10,20,30]` · `algorithm ['lloyd','elkan']` · `random_state [42]`
Agglomerative	`n_clusters [2–7 or 2–10]` · `linkage ['ward','complete','average','single']` · `metric ['euclidean','manhattan','cosine']` (ward → euclidean only) · `distance_threshold [0.5,1,5,10]`
DBSCAN	`eps [0.1,0.3,0.5,0.7,1.0,1.5]` (scaled data) · `min_samples [3,5,10,15,20] or 2×D bracket` · `metric ['euclidean','manhattan','cosine']`

4.4 Ten common mistakes

1. **Not scaling** the data.
2. Choosing k by **lowest inertia** (always picks the biggest k).
3. Using `GridSearchCV` on **Agglomerative / DBSCAN** (no predict → crash).
4. Scaling **before** cross-validation instead of in a Pipeline (leakage).
5. Forgetting the `kmeans__` prefix in a Pipeline grid.
6. Combining **ward** with a non-euclidean metric, or ward with precomputed.
7. Setting both `n_clusters` and `distance_threshold`.
8. Tuning `random_state` (that is tuning luck).
9. Celebrating a high DBSCAN silhouette that comes from **90% noise**.
10. Trusting the score without **plotting** the clusters (silhouette favours round clusters).

4.5 Practice exercises

1. Run the K-Means template on `make_blobs(centers=6)`. Does silhouette find k=6? *(extend the k grid to 2–10)*
2. Remove `StandardScaler` from the pipeline on columns of very different scale. What changes? *(compare ARI or the plot)*
3. Run the hierarchical loop on `make_circles`. Which linkage wins by silhouette, and which is actually right? *(plot both)*
4. Use the k-distance plot to choose eps for DBSCAN on your own data, then grid-search around it. *(try min_samples = 2×D)*
5. Build data with 2 densities and compare DBSCAN vs HDBSCAN. *(`from sklearn.cluster import HDBSCAN`)*

APPENDIX

Glossary

Term	Meaning
Centroid	The mean point (centre) of a cluster in K-Means.
Inertia / SSE	Sum of squared distances from points to their centroid; lower = tighter.
Silhouette score	−1…+1 measure of how well each point fits its own cluster vs the nearest other one.
Calinski-Harabasz	Between-cluster ÷ within-cluster dispersion; higher = better.
Local minimum	A stable-looking but poor solution the algorithm gets stuck in.
Convergence	The moment centroids stop moving (change < `tol`).
Dendrogram	Tree diagram of the merge history in hierarchical clustering.
Linkage	Rule for distance between two *clusters*.
Chaining	Distinct clusters merged through a thin bridge of points (single linkage, large eps).
Transductive	Only labels the data it was fitted on; no `.predict()`.
Core / border / noise	DBSCAN point types: dense centre / edge of a cluster / outlier (−1).
eps	Radius of the DBSCAN neighbourhood circle.
Curse of dimensionality	In many dimensions all points look equally far apart.
Data leakage	Test-fold information accidentally influencing training (e.g., scaling before splitting).
ARI	Adjusted Rand Index: agreement with true labels (1 = perfect, 0 = random).
PCA	Compresses many columns into a few that keep most of the variation.

Complete code templates (your originals)

These are your three templates exactly as supplied, for easy copy-paste. Use the walkthrough tables in each Part to understand them.

A · K-Means + Pipeline + GridSearchCV

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs
```

```python
# 1. Generate synthetic data (replace 'X' with your actual DataFrame)
X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0, random_state=42)

# 2. Define a custom scorer for Unsupervised Learning
# GridSearchCV defaults to 'accuracy' which fails without target labels (y).
# We build a custom wrapper to calculate the Silhouette Score instead.
def silhouette_scorer(estimator, X, y=None):
    labels = estimator.predict(X)
    # Silhouette score requires at least 2 clusters to compute mathematically
    if len(set(labels)) < 2:
        return -1
    return silhouette_score(X, labels)

# 3. Build a pipeline
# Scaling is strictly required for distance-based algorithms like K-Means.
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('kmeans', KMeans(random_state=42))
])

# 4. Define the hyperparameter grid
# The 'kmeans__' prefix is required to tell the Pipeline which step to tune.
param_grid = {
    'kmeans__n_clusters': [2, 3, 4, 5, 6, 7],
    'kmeans__init': ['k-means++', 'random'],
    'kmeans__n_init': [10, 20],
    'kmeans__algorithm': ['lloyd', 'elkan']
}

# 5. Configure and run GridSearchCV
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring=silhouette_scorer,
    cv=3,        # Splits data into 3 folds to validate cluster stability
    n_jobs=-1    # Uses all available CPU cores to speed up execution
)

# Fit the grid search to the data (no 'y' variable needed)
print("Running Grid Search...")
grid_search.fit(X)

# 6. Extract results
print(f"Best Silhouette Score: {grid_search.best_score_:.4f}")
print("Best Parameters:", grid_search.best_params_)

# 7. Apply the optimized model
best_model = grid_search.best_estimator_
cluster_assignments = best_model.predict(X)

# Optional: Add the cluster labels back to a pandas DataFrame
# df = pd.DataFrame(X)
# df['Cluster'] = cluster_assignments
```

B · Hierarchical + ParameterGrid

```python
import pandas as pd
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_moons

# 1. Generate synthetic data
X, _ = make_moons(n_samples=500, noise=0.1, random_state=42)

# 2. Scale the data (Mandatory for distance-based clustering)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Define the hyperparameter grid
```

```python
# Note: If 'ward' linkage is tested, metric MUST be 'euclidean'
param_grid = {
    'n_clusters': [2, 3, 4, 5, 6],
    'linkage': ['ward', 'complete', 'average', 'single'],
    'metric': ['euclidean', 'manhattan']
}

# 4. Initialize tracking variables
best_score = -1.0
best_params = None
best_labels = None

# 5. Execute the Grid Search manually using ParameterGrid
print("Running Grid Search...")
for params in ParameterGrid(param_grid):

    # Catch the mathematical incompatibility between Ward and non-Euclidean metrics
    if params['linkage'] == 'ward' and params['metric'] != 'euclidean':
        continue

    try:
        # Initialize and fit the model
        model = AgglomerativeClustering(**params)
        labels = model.fit_predict(X_scaled)

        # Calculate the score (requires at least 2 clusters and less than N clusters)
        if 1 < len(set(labels)) < len(X_scaled):
            score = silhouette_score(X_scaled, labels)

            # Update best score
            if score > best_score:
                best_score = score
                best_params = params
                best_labels = labels

    except Exception as e:
        # Catch any other incompatible parameter combinations
        pass

# 6. Output the optimal configuration
print(f"Best Silhouette Score: {best_score:.4f}")
print(f"Optimal Hyperparameters: {best_params}")

# Optional: Attach optimal labels back to the dataset
# df = pd.DataFrame(X)
```

C · DBSCAN + ParameterGrid

```python
import numpy as np
import pandas as pd
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

# 1. Generate synthetic data
X, _ = make_blobs(n_samples=1000, centers=4, cluster_std=0.8, random_state=42)

# 2. Scale the data (Mandatory for eps to function logically)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Define the hyperparameter grid
param_grid = {
    'eps': [0.1, 0.3, 0.5, 0.7, 1.0, 1.5],
    'min_samples': [3, 5, 10, 15, 20]
}

# 4. Initialize tracking variables
best_score = -1.0
best_params = None
```

```python
best_labels = None

# 5. Execute the Grid Search manually
print("Running Grid Search for DBSCAN...")
for params in ParameterGrid(param_grid):

    # Initialize and fit the model (using all CPU cores)
    model = DBSCAN(**params, n_jobs=-1)
    labels = model.fit_predict(X_scaled)

    # Identify how many actual clusters were found, ignoring noise (-1)
    unique_labels = set(labels)
    n_clusters = len(unique_labels) - (1 if -1 in labels else 0)

    # Silhouette score mathematically requires at least 2 clusters
    if n_clusters >= 2:
        # Calculate score using only the non-noise points
        # Optional: You can penalize the score if too much data is classified as noise
        score = silhouette_score(X_scaled, labels)

        if score > best_score:
            best_score = score
            best_params = params
            best_labels = labels

# 6. Output the optimal configuration
if best_params is not None:
    print(f"Best Silhouette Score: {best_score:.4f}")
    print(f"Optimal Hyperparameters: {best_params}")

    # Calculate noise percentage for the best model
    noise_ratio = list(best_labels).count(-1) / len(best_labels) * 100
    print(f"Percentage of data classified as noise: {noise_ratio:.2f}%")
else:
    print("No parameter combination found at least 2 distinct clusters.")
```

Created for Tamzid · all experiments run with scikit-learn 1.9.1 · good luck with your ML journey!